“DESIGN & IMPLEMENTATION OF A CLOUD - BASED ROBOTIC ARM TELEOPERATION SYSTEM USING FPGA SOC”

A Project Report Submitted in the partial fulfillment of requirement of the Degree of

Bachelor of Technology

in

Electronics & Telecommunication Engineering

to

PRIYADARSHINI COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

Under the Guidance of

Guide

Dr. (Mrs).P.U.Chati

Assistant Professor, Department of Electronics & Telecommunication Engineering

Submitted by

Ketan Shrirao (135) **Sagar Mallik (145)**

Hemant Pise (132) **Ayush Bobde (127)**

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR-440019

2026-2027

VISION OF THE INSTITUTE

To become one of the India's leading engineering institutes in both education and research. We are committed to provide quality and state of the art technical education to our student, so that they become technologically superior and in turn contribute for creating a great society.

MISSION OF THE INSTITUTE

i) Fostering a dynamic learning environment that equips students with technical expertise, problem-
solving skills and a deep commitment to ethical practices.

ii) To cultivate a culture of innovation, incubation, research and entrepreneurship that drives
technological advancements.

iii) To uphold the spirit of mutual excellence while interacting with stake holders of our Institutional
ecosystem.

iv) Promoting lifelong learning, professional growth and ensuring holistic development of students and the well being of society.

VISION OF THE DEPARTMENT

To achieve excellence in academics and research by imparting quality education in the rapidly changing field of Electronics and Telecommunication Engineering.

MISSION OF THE DEPARTMENT

M1. To educate the students with the state of art technologies to meet the growing challenges of the
industries and educational institutions.

M2. To equip the students with strong foundations to enable them for continuing education and research.

M3. Provide ethical and value based education by promoting activities addressing the social needs.

PEOs , POs and PSOs

Program Educational Objectives of the Department as below:

PEO-1. To provide graduates with a solid foundation in mathematical, scientific and Engineering fundamentals required to formulate, analyze & solve complex Engineering problems.
PEO-2. To create graduates with technical expertise to handle multi-disciplinary projects and with ability to excel in professional career and/or higher education.
PEO-3. To inculcate leadership qualities among graduates with ethical responsibilities and global security awareness.
PEO-4. To develop graduates with capability to apply knowledge and evaluate results based on facts, experimentation, research and exploring to new simulation tools.

Program Outcomes of the Department as below:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety,whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large,such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes of the Department as below:

1) To identify, formulate and analyze the problems in Electronics & Telecommunication Engineering by using principles of Mathematics and Engineering fundamentals.
2) Applying Electronic and Communication Engineering knowledge for design and analysis of various system.

PRIYADARSHINI COLLGE OF ENGINEERING, NAGPUR

Department of Electronics & Telecommunication Engineering

CERTIFICATE

This is to certify that project report entitled **"Design & Implementation of a Cloud - based Robotic ARM Teleoperation System using FPGA SoC"** is a bonafide work done by the students **Ketan Shrirao, Hemant Pise, Sagar Mallik, Ayush Bobde.**

The project report is submitted to **PRIYADARSHINI COLLEGE OF ENGINEERING NAGPUR,** an Autonomous Institute Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur in partial fulfillment of the requirements for the degree of Bachelor of Technology in (Name of Department) for Session 2026-2027

GUIDED BY

Dr. (Mrs).P.U.Chati

Dr.V.K.Taksande
Head of Department

Dr. S.A. Dhale
Principal,PCE

PRIYADARSHINI COLLGE OF ENGINEERING, NAGPUR-440019 (INDIA)

Department of Electronics & Telecommunication Engineering

DECLARATION

We, the undersigned, declare that the project entitled **"Design & Implementation of a Cloud - based Robotic ARM Teleoperation System using FPGA SoC",** being submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in **Electronics & Telecommunication Engineering Department,** Priyadarshini College of Engineering Nagpur, an Autonomous Institute Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur is the work carried out by us.

Place: Nagpur

Date:

PROJECTEES:

1] Ketan Shrirao

2] Hemant Pise

3] Sagar Mallik

4] Ayush Bobde

ACKNOWLEDGEMENT

It is our pleasure to acknowledge our sincere thanks with a deep sense of gratitude towards our project guide **"Dr.P.U.Chati"**, Assistant Proffessor, Electronics & Telecommunication Engineering Department for her continuous knowledge and support in conducting this dissertation work. She has a whole heartedly helped us in this endeavor at all the stages of this work.

We would like to express our sincere gratitude to **Dr. S. A. Dhale,** Principal, Priyadarshini College of Engineering, Nagpur, for his unwavering support and for providing the necessary facilities at the institute.

Our heartfelt thanks also go to **Dr. G. M. Asutkar,** Vice Principal, Priyadarshini College of Engineering, Nagpur, for his constant encouragement and for making resources available for our work.

We are deeply grateful to **Dr.V.K.Taksande,** Head of the Department of Electronics & Telecommunication Engineering, Priyadarshini College of Engineering, Nagpur, for his invaluable guidance and for offering the necessary support throughout this project.

We also take the opportunity to thank all, who have directly or indirectly extended help and encouragement in executing this project.

PROJECTEE:

1] Ketan Shrirao

2] Hemant Pise

3] Sagar Mallik

4] Ayush Bobde

ABSTRACT

When human technicians and emergency personnel have to work in unpredictable or unreachable places—like neutralizing unexploded improvised explosive devices (IEDs) checking radioactive containment vessels in decommissioned nuclear plants or stopping high-pressure toxic chemical leaks rigid pre-programmed robotic automation is not enough. In these situations, there is no substitute for human cognitive intuition, careful touch and split-second tactical thinking. This truth shows why robotic teleoperation is so important: it lets human motor skills travel over distance while keeping the operator safe. However, building a teleoperation system over the open internet is very hard. Standard microcontrollers cannot handle high-frequency network packets and many motor pulses at the time adding 10 to 50 µs of software timer interrupt jitter that makes motors chatter gears wear and precision servos overheat. Embedded Linux also has driver crashes and power rail drops when using USB Wi-Fi dongles.

This project shows the design, hardware creation and real-world tests of a "Cloud-Based 6-DOF Robotic ARM Teleoperation System using FPGA SOC". The main idea is separation of functions: high-level network routing, security and cloud sync go to an embedded Linux OS on an ARM Cortex-A9 Hard Processor System (HPS) while low-level hard real-time motor control and analog signal capture go to dedicated parallel digital logic gates built on the FPGA part of an Intel/Altera Cyclone V SE System-on-Chip (DE10-Nano). At the master station a comfortable dual-axis analog joystick console (JS 1 and JS 2) records operator speed setpoints. Four continuous analog voltage lines are read by an onboard 12-bit LTC2308 SAR ADC through a custom Verilog SPI master controller. The ARM Linux HPS reads the digitized coordinate words from the lightweight HPS-to-FPGA AXI Bridge (base address 0xFF200000) using virtual memory mapping and sends JSON telemetry frames at 20 Hz to Google Firebase Realtime Database.

To avoid the driver problems of USB Wi-Fi dongles on embedded Linux we built a hardware gateway with an ESP32 microcontroller and a WIZnet W5500 SPI Ethernet module running LwIP NAPT firmware.This gateway connects 4G/5G cellular hotspots straight to the DE10-Nano's Gigabit Ethernet port (eth0) with less than a millisecond local switch delay. At the slave station a second DE10-Nano board checks the cloud telemetry. Writes calibrated cycle counts to six dedicated 20-bit hardware PWM generators running at 50 MHz in FPGA logic driving heavy-duty TowerPro MG996R metal-gear servos (Base, Shoulder, Elbow) and MG90S micro-servos (Wrist Pitch, Wrist Roll, Claw) with zero timer jitter.

Tests show a closed-loop teleoperation latency of 77.2 ms over cellular networks, joint tracking accuracy within ±1.4° repeatability of ±0.52° and thermally stable, jitter-free holding torque, under 7.65 A peak dynamic current. The project delivers a working, open-architecture cyber-physical platform that unites FPGA hardware determinism and modern cloud robotics.

INDEX

CHAPTER 1: INTRODUCTION	**1**
1.1 BACKGROUND	**2**
1.2 PROBLEM STATEMENT	**3**
1.3 OBJECTIVE	**4**
1.4 SIGNIFICANCE	**5**
1.5 SCOPE	**5**
CHAPTER 2: LITERATURE REVIEW	**6**
2.1 REVIEW OF LITERATURE	**7**
2.2 SUMMARY OF LITERATURE SURVEY	**9**
CHAPTER 3: LITERATURE REVIEW	**10**
3.1 INTRODUCTION	**11**
3.2 BLOCK DIAGRAM & CIRCUIT DIAGRAM OF DEVELOPED SYSTEM	**12**
3.3 HARDWARE DEVELOPMENT	**16**
3.4 OVERALL SYSTEM WORKING STAGES	**20**
CHAPTER 4: SOFTWARE/HARDWARE REQUIRED	**21**
4.1 LIST OF COMPONENTS	**22**
4.2 DESCRIPTION OF THE COMPONENT	**23**
4.3 SOFTWARE DEVELOPMENT TOOL & OPERATING ENVIRONMENT	**32**
CHAPTER 5: RESULTS AND DISCUSSIONS	**33**
5.1 EXPERIMENTAL TEST AND SETUP	**34**
5.2 SYSTEM LATENCY AND TIMING BREAKDOWN ANALYSIS	**34**
5.3 NETWORK JITTER AND PACKET DELIVERY	**35**
5.4 PICTURE OF DEVELOPED SYSTEM	**36**

CHAPTER 6: CONCLUSIONS, FUTURE SCOPE APPLICATIONS & REFERENCES 37

6.1 CONCLUSION 38

6.2 FUTURE SCOPE 39

6.3 REFERENCES 40

APPENDIX 41-82

CODES

CERTIFICATES OF PAPER PUBLICATION

DATASHEETS

PHOTO WITH DETAILS

PHOTO WITH GUIDE

CO-PO and PSO MAPPING

PLAGIARISM REPORT

TABLE OF CONTENTS

Sr no.	Figure Name	Page no.
1	One Page Poster	55
2	Datasheets	56
	(A) DE10-Nano Development Kit	56
	(B) ESP-32 Series	66
	(C) W5500 Datasheet	71

LIST OF TABLES

Sr no.	Figure Name	Page no.
2.1	State of the Art Comparative Synthesis Matrix	9
3.1	Memory mapping & Address mapping	18
3.2	Hardware Interfacing Signal Mapping across Transmitter and Receiver Nodes	19
4.1	List of Hardware and Electronic Components Used in the Developed System	22
4.2	Table joint Mapping Configuration	26
5.1	Network Transmission Quality Comparison: Wired Cat5e vs. 2.4 GHz Wi-Fi	35

LIST OF FIGURES

Sr no.	Figure Name	Page no.
3.1	System Architecture and Data Flow of the Cloud-Based Robotic Arm Teleoperation Platform	12
3.2	Circuit Diagram of Transmitter End	13
	Circuit Diagram of Receiver End	15
4.1	Terasic DE10-Nano Cyclone V FPGA-SoC Development Board Hardware	24
4.2	Dual-Axis Analog Joystick Module	25
4.3	ESP32-WROOM 32 Development Kit Microcontroller Board	27
4.4	WIZnet W5500 SPI Ethernet Module	28
4.5	TowerPro MG996R & MG90S Servomotors	28-29
4.6	6 DOF Robotic Arm	30
4.7	Cat5e Network Cable	31
5.1	Final Developed System	36

LIST OF SYMBOLS, ABBREVIATIONS

Abbreviation	Expanded Description
ADC	Analog to Digital Converter
ALU	Arithmetic Logic Unit
ARM	Advanced RISC Machines
AXI	Advanced eXtensible Interface (ARM AMBA Interconnect)
BOM	Bill of Materials
CPU	Central Processing Unit
DC	Direct Current
DetNet	Deterministic Networking (IETF Working Group)
DH / D-H	Denavit-Hartenberg Kinematic Notation
DIP	Dual In-line Package (Switches)
DOF	Degree of Freedom
DSP	Digital Signal Processor
ESD	Electrostatic Discharge
ETC	Electronics and Telecommunication Engineering
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
GPIO	General Purpose Input / Output
HDL	Hardware Description Language
HPS	Hard Processor System (Cyclone V Dual ARM Cortex-A9)
I2C	Inter-Integrated Circuit Serial Bus
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
IP	Internet Protocol / Intellectual Property
JS	Joystick (JS 1 and JS 2)
JSON	JavaScript Object Notation
LE	Logic Element (FPGA Configurable Core Block)
LDO	Low Dropout Voltage Regulator
LUT	Look-Up Table
LwIP	Lightweight IP (TCP/IP Stack)
MAC	Media Access Control
MMU	Memory Management Unit
NAPT	Network Address Port Translation
NVS	Non-Volatile Storage
OS	Operating System
PCB	Printed Circuit Board

PHY	Physical Layer Transceiver
PIO	Parallel Input / Output (Avalon Memory-Mapped Core)
PISO	Parallel-In-Serial-Out Shift Register
PWM	Pulse Width Modulation
Qsys	Intel Platform Designer System Integration Tool
RBF	Raw Binary File (FPGA Configuration Bitstream)
RC	Radio Controlled / Remote Control
RTL	Register Transfer Level
RTT	Round Trip Time
SAR	Successive Approximation Register
SIPO	Serial In Parallel Out Shift Register
SMPS	Switched-Mode Power Supply
SoC	System on Chip
SPI	Serial Peripheral Interface
SSE	Server-Sent Events
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TSN	Time-Sensitive Networking (IEEE 802.1 Standard)
UART	Universal Asynchronous Receiver Transmitter
UDP	User Datagram Protocol
UTP	Unshielded Twisted Pair
VCC	Voltage Common Collector (Positive Supply Rail)
VRx / VRy	Variable Resistor X-Axis / Y-Axis (Analog Voltage)

CHAPTER – 1

INTRODUCTION

1.1 BACKGROUND

Robotics has changed automated manufacturing, electronics assembly and warehouse fulfilment a lot. In predictable factory cells pre-set robotic arms perform repeat pick-and-place tasks with less than a millimetre of error and move faster than people can. When Robotics goes outside the neat factory floor into chaotic, dangerous or ever-changing places fully autonomous control systems fail badly. In cases like disarming an IED stopping a toxic chemical valve in a plant shutting down radioactive cooling lines in damaged nuclear reactors or doing surgery in quarantine wards the surroundings are wild and unpredictable. Vision and path-planning algorithms have trouble when faced with tangled wires, half-crushed stone shifting loads and soft toxic substances. In these high-stakes jobs if algorithms fail there can be explosions, pollution or death of people. On the hand sending people into these places puts them at risk of heat, chemicals or radiation. This big problem shows why robotic teleoperation is so needed. Teleoperation moves thinking, sense and skilled handwork over distance and keeps the human operator safe. In the past robotic teleoperation started with wired master-slave manipulators.

Early in nuclear hot-cell places an operator held a pantograph or many analog joysticks that were wired straight to the far arm using heavy shielded copper cables. These analog setups gave predictable answers over five to ten metres but they were limited by cable loss, noise, snagging on obstacles and the fact that they could not work over cities, borders or far disaster spots. To get past wired limits modern robotics now uses packet-switched network teleoperation using the Internet of Things and flexible cloud computing. In theory sending robot data through central cloud databases lets robots work anywhere. An expert at a command can control a special tele-robot thousands of kilometres away using ordinary 4G/5G or fiber networks. However, moving from a copper wire to cloud teleoperation brings serious engineering problems. The biggest problem comes from the fact that cloud networking's not predictable while servo motors need strict real-time timing [9].

Embedded Processing Bottleneck: embedded computers like 8-bit AVR chips or single-core 32-bit ARM Cortex-M processors run code in a straight line. When one core tries to do things at once TCP/IP networking reading JSON packets handling HTTP interrupts reading many joysticks and making precise PWM signals for six motors software timer’s clash. If the timing jitters by 20 to 50 microseconds in a normal 50 Hz servo signal the motors shake gears wear out fast heat builds up and the motor can lose its grip. If cloud latency changes or the OS thread waits for a socket the robot arm. May drop dangerous loads. This project, titled "**Design & Implementation of a Cloud-based 6-DOF Robotic ARM Teleoperation System using FPGA SOC**" fixes these tangled problems by using a cyber-physical hardware-software design on the Intel/Altera Cyclone V System-on-Chip FPGA. The main idea of this design is to separate network control, which's unpredictable from motor control, which must be exact. By combining a dual-core ARM Cortex-A9 HPS running Linux with a reconfigurable FPGA, on one chip the system gives cloud communication to the HPS and hands very fast servo pulse creation and SPI ADC reading to special Verilog logic running at 50 MHz [3].

1.2 PROBLEM STATEMENT

Problem Statement: To create, build, model, make and test a fast no delay, connected to the cloud 6-DOF robotic arm remote control system that completely separates the time delays from the cloud network from the real-time movement of the motors using a special FPGA chip that does many things at once.

The main engineering problems that this problem statement tries to solve are:

1. Real-Time Jitter Elimination: Getting rid of the timing problems in the software that controls the motors so that the robot parts don't shake and break. This is done without using special tools that companies usually use.

2. Cloud-to-Hardware Co-Design: Making a way to connect the cloud with the robot hardware so that data moves quickly and smoothly. This is done using a connection that links the computer software with the robot parts at the speed of the clock.

3. Reliable Cloud Telemetry Pipeline: Making a fast way to send information between the robot and the cloud. This is done by avoiding the parts of the computer's Wi-Fi and using a service called Google Firebase to keep the robot and the cloud in sync in less, than 100 milliseconds.

4. Intuitive Multi-DOF Master Interface: Making a control stick that's easy to use and lets one person move all six parts of the robot without getting confused or tired. This control stick has two joysticks. Can switch between different modes.

5. Power Integrity and Electrical Isolation: Making a power system that can handle a lot of electricity and protect the robot from problems. This system keeps the robot from getting damaged by power surges and makes sure the robot's computer parts work properly.

1.3 OBJECTIVES

The primary technical objectives formulated for this undergraduate project are as follows:

1. To construct a master teleoperation console using two KY-023 2-axis analog thumb joysticks (JS 1 and JS 2) with built-in tactile pushbuttons enabling velocity-based control of six degrees of freedom in the robotic system.

2. To. Implement a dedicated 12-bit LTC2308 SPI master ADC controller in Verilog HDL on the Cyclone V FPGA ensuring microsecond-deterministic acquisition of analog signals from the joysticks.

3. To develop speed, zero-copy communication between hardware and software across the Lightweight HPS-to-FPGA AXI Bridge using direct Linux user space memory mapping via base address as 0xFF200000.

4. To build a hardware-assisted Layer-3 Wi-Fi-to-Ethernet NAT router using an ESP32-WROOM-32 microcontroller and a WIZnet W5500 SPI Ethernet module running LwIP NAPT firmware.

5. To establish a noise- Cat5e wired Ethernet link between the W5500 module and the DE10-Nano Gigabit Ethernet port (eth0) assigning static IP addresses (192.168.137.50 and 192.168.137.55) avoiding reliance on Linux USB Wi-Fi dongles.

6. To deploy an event-driven cloud telemetry pipeline on Google Firebase Realtime Database located in Singapore (asia-southeast1) synchronizing six-joint coordinate data at a rate of 20 Hz.

7. To synthesize six 20-bit hardware PWM generators inside FPGA logic clocked at 50 MHz generating jitter-free 50 Hz control waveforms calibrated for standard servo pulses (1.0–2.0 ms) and extended pulse widths.

8. To. Operate an articulated 6-DOF metallic robotic arm equipped with three TowerPro MG996R metal-gear servos (Base, Shoulder, Elbow) and three MG90S micro-servos (Wrist Pitch, Wrist Roll, Gripper Claw).

9. To implement a power distribution network powered by a regulated 5V 10A DC SMPS using an equipotential star-ground topology to eliminate floating-ground noise and ensure signal integrity.

10. To perform by-joint mechanical calibration re-indexing the center splines and applying software deadband filtering set to 100 counts reducing mechanical end-stop binding and minimizing drift.

11. To maintain closed-loop teleoperation latency strictly below 150 ms across commercial 4G and 5G mobile networks meeting human factors thresholds for real-time control.

12. To experimentally validate tracking accuracy within ±1.4° angular repeatability within ±0.52° and current consumption, under high-torque dynamic plan ensuring reliable performance during operation.

1.4 SIGNIFICANCE

Multidisciplinary Impact: The importance of this research is clear in both social areas. It has effects in fields like robotics, computing and industrial safety:

Life Safety in Hazardous Interventions: It allows human bomb disposal technicians teams that deal with dangerous chemicals and workers who clean up after nuclear accidents to do difficult physical tasks from safe command bunkers that are hundreds of miles away. This removes the danger to their lives.

Architectural Paradigm for Cloud Robotics: It shows a model that can be copied and used again. This model is open. Works well with both cloud systems and real-time control of robots. It solves a standing problem without needing expensive special hardware [5].

Actuator Longevity and Thermal Reliability: By using 50 MHz FPGA counters that are locked in hardware to create motor control pulses the system has no phase jitter. It has a precision of 0.02 µs. This stops the problems and the heat problems that often happen with robotic arms that use microcontrollers.

Economic and Educational Accessibility: The system uses parts like DE10-Nano, ESP32 MG996R servos and Google Firebase. It gives the performance as expensive industrial systems but at a much lower cost, than commercial aerospace telemanipulations.

1.5 SCOPE

System Boundaries: To keep the engineering focus tight the limits, functional scope and operating constraints of this project are clearly defined.

Kinematic Scope: A 6 degree of freedom arm that moves at Base Azimuth (Joint 1 0°–180°), Shoulder Elevation (Joint 2 0°–180°), Elbow Flexion (Joint 3 0°–180°), Wrist Pitch (Joint 4 0°–180°), Wrist Roll (Joint 5 0°–180°) and an articulating Parallel-Jaw Claw Gripper (Joint 6 0°–180°) [8].

Payload Capacity: Built for light to tasks in a remote lab the arm can lift 250 grams when fully stretched out (380 mm) and up to 450 grams when pulled back close (200 mm).

Control Topology: One-way teleoperation that sends commands from a master to a slave while logging telemetry data. Two-way haptic feedback and visual servoing are planned for research.

Network Constraints: Works over public TCP/IP networks using Google Firebase Web Sockets. The arm is designed to run on typical 4G or 5G mobile hotspots keeping round-trip delays, below 100 milliseconds.

Hardware Power Domain: The arm's motors run on a 5-volt DC, 10-amp switched-mode power supply. The control boards (DE10-Nano and ESP32) use their regulated logic power and are connected through a common star-ground rail.

CHAPTER – 2

LITERATURE SURVEY

2.1 REVIEW OF LITERATURE

A literature has been carried out on various types of work done on the topieor the related topics in the previous few years. The survey is as shown below:

1: FPGA Used for Reducing Delay in the Tactile Internet [3]

Authors: J. C. V. S. Junior, S. N. Silva, M. F. Torquato, T. Mahmoodi, M. Dohler and M. A. C. Fernandes (Sensors, vol. 22, No. 20 P. 7851, 2022)

The Tactile Internet aims to allow time physical interaction and the sending of touch information over wireless networks, which needs delays that are always less than 1 millisecond. When using software-based systems on microprocessors there are problems with unpredictable delays and slow processing. Junior and his team solved this problem by using digital circuits inside an FPGA. Their design makes it faster to process packets read protocols and use control methods. The authors showed that using hardware made the processing time very short. Even though their study was done in environments their results explain why we use our system: moving important tasks like creating 50 Hz PWM signals and collecting 12-bit SPI data to the FPGA avoids delays, from the CPU and gives perfect timing [3].

2: Design and Implementation of UART using Verilog: Parallel-in-Serial-Out and Serial-in-Parallel-Out Modules [2]

Authors: P. Chati, K. Shrirao, H. Pise, S. Mallik and A. Bobde (International Journal of Advanced Research. IJAR, vol. 13 No. 7 Pp. 924–928, Jul. 2025)

Authored by project supervisor Dr. P. U. Chati and the project student team at Priyadarshini College of Engineering, Nagpur this paper details the design, simulation and hardware synthesis of a Universal Asynchronous Receiver-Transmitter (UART) core using Verilog HDL. The authors specifically implemented Parallel-In-Serial-Out (PISO) shift registers for data transmission and Serial-In-Parallel-Out (SIPO) shift registers for data reception. A dedicated baud-rate generator divides the system master clock to produce transmission rates (115,200 baud) while finite state machines (FSMs) manage start-bit detection, data latching and stop-bit validation. In the context of our teleoperation roadmap this publication represents the milestone (Phase 1: UART hardware link between two FPGAs). Validating register-transfer-level (RTL) transmission, between two isolated DE10-Nano boards established the digital communication channel before escalating to embedded Linux networking and AXI memory mapping [2].

3: An FPGA-Based Controller for Collaborative Robotics [7]

Authors: B. P. Jeppesen, N. Roy, L. Moro and F. Baronti (IEEE 26th International Symposium on Industrial Electronics. ISIE, pp. 1067–1072, 2017)

Collaborative robots (cobots) work in shared workspaces with human workers requiring strict functional safety, high electrical efficiency and quick response for motor control. Jeppesen et al. Created an FPGA-based controller that can run low-level motor commutation track joint position and velocity sense current at high frequencies and handle safety-related collision prevention at multi-kilohertz sampling rates. By running these feedback loops in parallel hardware logic blocks the controller avoided CPU thread conflicts and made sure safety deadlines were always met. Even though Jeppesen et al. Worked on industrial drives their main method shows that our approach is valid: building specific hardware-timed PWM generators in FPGA logic to control all six joints of our robotic arm at the same time without any timing issues, between channels [7].

4: Deterministic Networking Empowered Robotic Teleoperation [1]

Authors: C. Yang, H. Yu, Q. Guo, T. Taleb, J. C. Requena and K. Tammi (IEEE Network, vol. 39 No. 4 Pp. 280–289, Jul. 2025)

In teleoperation across normal IP and wireless networks random packet queuing, route changes and sudden traffic spikes create jitter that is not deterministic. This jitter can make control loops unstable. Can even damage robots. Yang and colleagues looked at how to use Networking (TSN) standards and IETF Deterministic Networking designs together with edge computing orchestration. Their goal was to keep delay within a fixed envelope and to avoid any packet loss. The results show that deterministic edge packet scheduling can keep remote robotic control loops. Even though carrier-grade TSN is not yet available, on cellular links their theory helped shape our own design. We built a self-contained router using an ESP32 and a W5500 SPI Ethernet Layer-3 NAT. This router bypasses Linux USB Wi-Fi dongles and works with Google Firebase Realtime Database SSE/WebSocket streaming. The result is a low-overhead bounded-latency telemetry system that works over networks [1].

2.2 SUMMARY OF LITERATURE SURVEY

Table 2.1 shows a comparison of the nine papers that were reviewed. It looks at the design of each paper the way they communicate how they move how fast they work and the problems they have. This comparison is made against our Cloud-Based 6-DOF FPGA-SoC Teleoperation System.

Table 2.1: State of the Art Comparative Synthesis Matrix

Ref. & Authors	Hardware Core	Communication Protocol	Actuation / Control Mode	Latency Profile	Identified Technical Limitations
Junior et al. [3] (2022)	FPGA Logic Fabric	Hardware Tactile Internet Pipeline	Digital Predictive Control	Sub-millisecond (<1 ms)	Simulation-focused; lacked physical 6-DOF robotic manipulator with inductive loads.
Chati et al. [2] (2025)	Cyclone V FPGA	Asynchronous UART (Verilog)	PISO/SIPO Register Loopback	Microsecond (<10 µs)	Point-to-point serial link; lacked network routing and multi-joint motor drivers.
Jeppesen et al. [7] (2017)	Custom FPGA Board	Local Industrial Bus	Multi-kHz joint position loops	Real-time (<2 ms)	Confined to local industrial drives; no cloud or wireless teleoperation .
Yang et al. [1] (2025)	Edge Computing Nodes	TSN & DetNet Architecture	Remote teleoperation setpoints	Bounded (<20 ms)	Requires carrier-grade TSN infrastructure not available on public cellular links.

CHAPTER – 3

METHODOLOGY

3.1 INTRODUCTION

Engineering a cyber-physical system is fundamentally different from designing a purely software application or assembling a standalone electronic circuit on a workbench. In software an algorithm operates inside a mathematically abstract environment governed by clean memory allocations and deterministic processor clocks. In hardware physical linkages obey the laws of inertia, friction, gear backlash and cantilever bending moments. In power electronics inductive motor coils draw massive surges of electrical current that cause ground potential shifts and severe electromagnetic interference (EMI). Finally in networking, packet transmission across the public internet is subject to stochastic route fluctuations, carrier congestion and unpredictable signal attenuation.

The central challenge of system modelling in this project was therefore to harmonize four different physical and logical domains into a unified dependable operational loop:

(1) The analog domain of human joystick deflections (2) The microsecond digital domain of FPGA hardware logic gates, (3) The asynchronous interrupt-driven domain of an embedded Linux operating system and (4) The wide-area packet-switched domain of global cloud databases. Every component in this architecture was selected not as an off-the-shelf part but as a deliberate solution, to a specific failure mode encountered during early prototyping.

Then forcing an operator to wear clumsy drift-prone resistive flex sensor gloves that induce severe hand perspiration and non-linear calibration drift the master station utilizes an industrial-grade dual-axis analog joystick console (JS 1 and JS 2). Of relying on software timer interrupts on a microcontroller to pulse servo motors, an Intel/Altera Cyclone V FPGA fabric synthesizes dedicated 20-bit hardware counters operating from a 50 MHz crystal clock. Of battling the fragile crash-prone USB Wi-Fi drivers of embedded Linux, an autonomous ESP32 + W5500 SPI Ethernet Layer-3 NAT router provides a plug-and-play wired bridge to commercial 4G/5G cellular hotspots. This chapter details the system architecture providing the global block diagram, circuit schematics, bill of materials and exhaustive technical analyses of each hardware module and software tool.

3.2 BLOCK DIAGRAM & CIRCUIT DIAGRAM OF DEVELOPED SYSTEM

A) Block Diagram of the Prototype:

The end-to-end teleoperation platform uses a hierarchical master-slave architecture. The end-to-end teleoperation platform starts with a joystick that the Transmitter (Board A) reads. The Transmitter (Board A) turns the joystick deflections into digitised coordinates using an FPGA SPI ADC controller. The Linux HPS then reads those coordinates via an AXI bridge. The coordinates travel through a hardware ESP32 NAT gateway over 4G/5G links, to the Google Firebase Realtime Database. The end-to-end teleoperation platform is completed when the Receiver (Board B) polls the database. Writes the coordinates to FPGA PWM registers that drive the 6-DOF robotic arm.

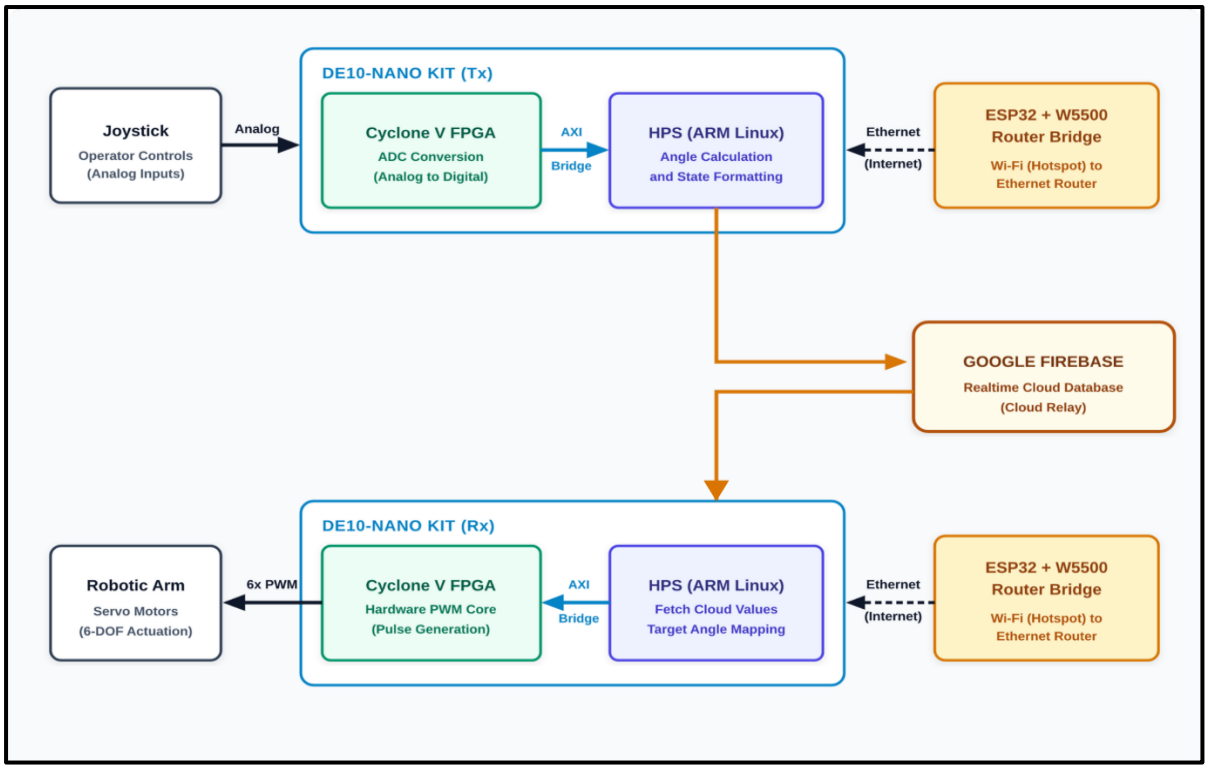


Fig 3.1: System Architecture and Data Flow of the Cloud-Based Robotic Arm Teleoperation Platform

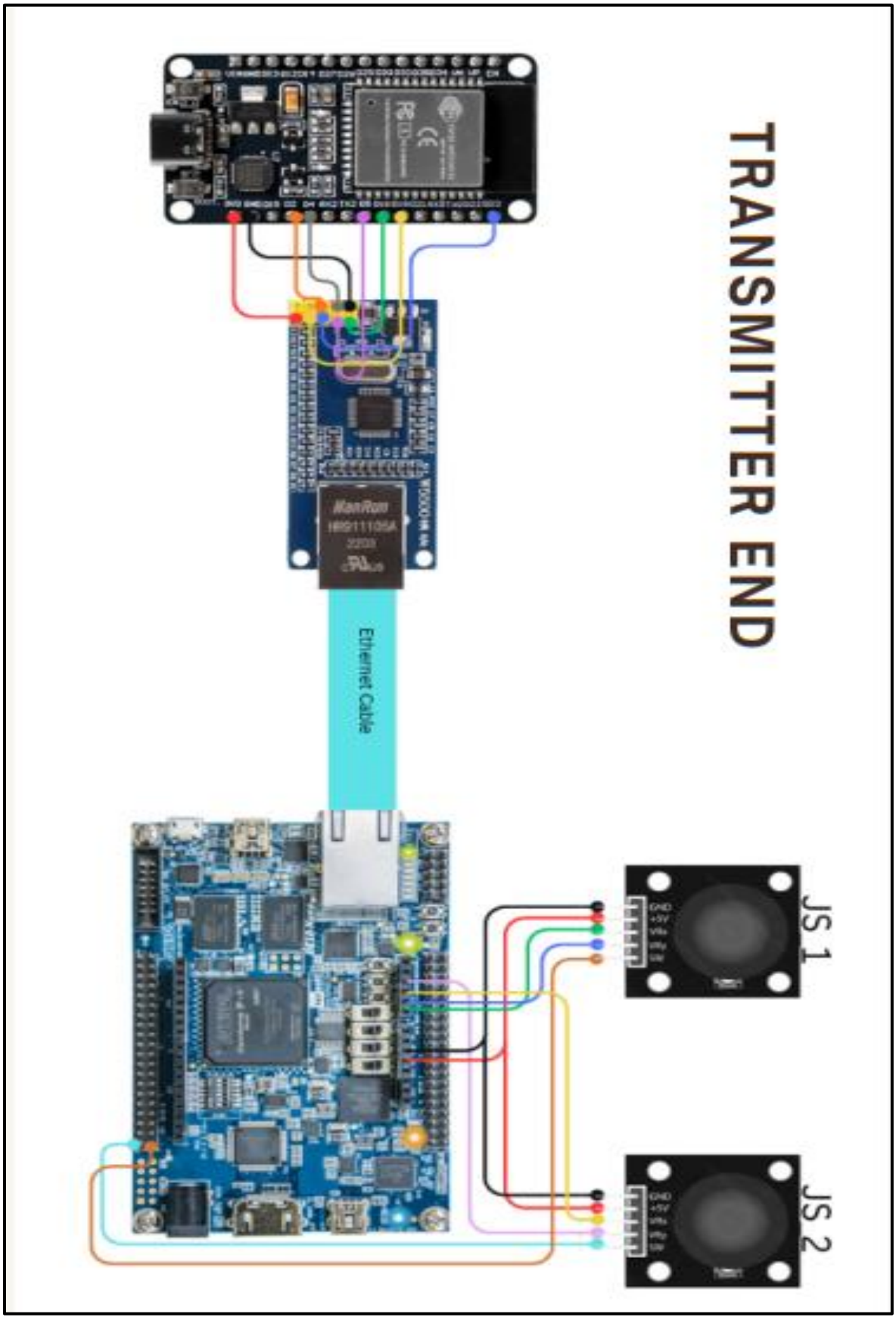


Fig 3.2 Circuit Diagram of Transmitter End

The system is split into two circuits: the Master Transmitter Circuit and the Slave Receiver Circuit as shown in the detailed schematics (Circuit Diagram Tx and Receiver End).

1. Transmitter End Circuit: includes the DE10-Nano board, two KY-023 analog joysticks (JS 1 and JS 2) an ESP32-WROOM-32 Development kit, a WIZnet W5500 SPI Ethernet module and a Cat5e patch cable that connects them. The joystick potentiometers receive power from the DE10-Nano 3.3V and GND pins. The four analog outputs (VRx, VRy of JS 1 and JS 2) go to Channels 0 1 2 and 3 on the ADC header on the board. The ESP32 is linked to the W5500 through a SPI bus: MOSI on GPIO23 MISO on GPIO19 SCK on GPIO18 CS on GPIO5. The W5500 RJ45 jack is plugged directly into the DE10-Nano Ethernet port.

2. Receiver End Circuit: uses an ESP32 and W5500 Ethernet gateway that is connected to the slave DE10-Nano board. Of joysticks the DE10-Nano Header JP1 on GPIO0 outputs six hardware PWM signals. These signals are:

1. Pin 3 (PIN_W12) to Base Servo
2. Pin 4 (PIN_D11) to Shoulder Servo
3. Pin 5 (PIN_D8) to Elbow Servo
4. Pin 6 (PIN_AH13) to Wrist Pitch Servo
5. Pin 7 (PIN_AF7) to Wrist Roll Servo
6. Pin 8 (PIN_AH14), to Gripper Claw Servo

All six servos are powered by a regulated 5V, 10A DC SMPS. The SMPS provides a bonding that connects the SMPS ground the servo ground leads and the DE10-Nano JP1 Pin 12 (GND).

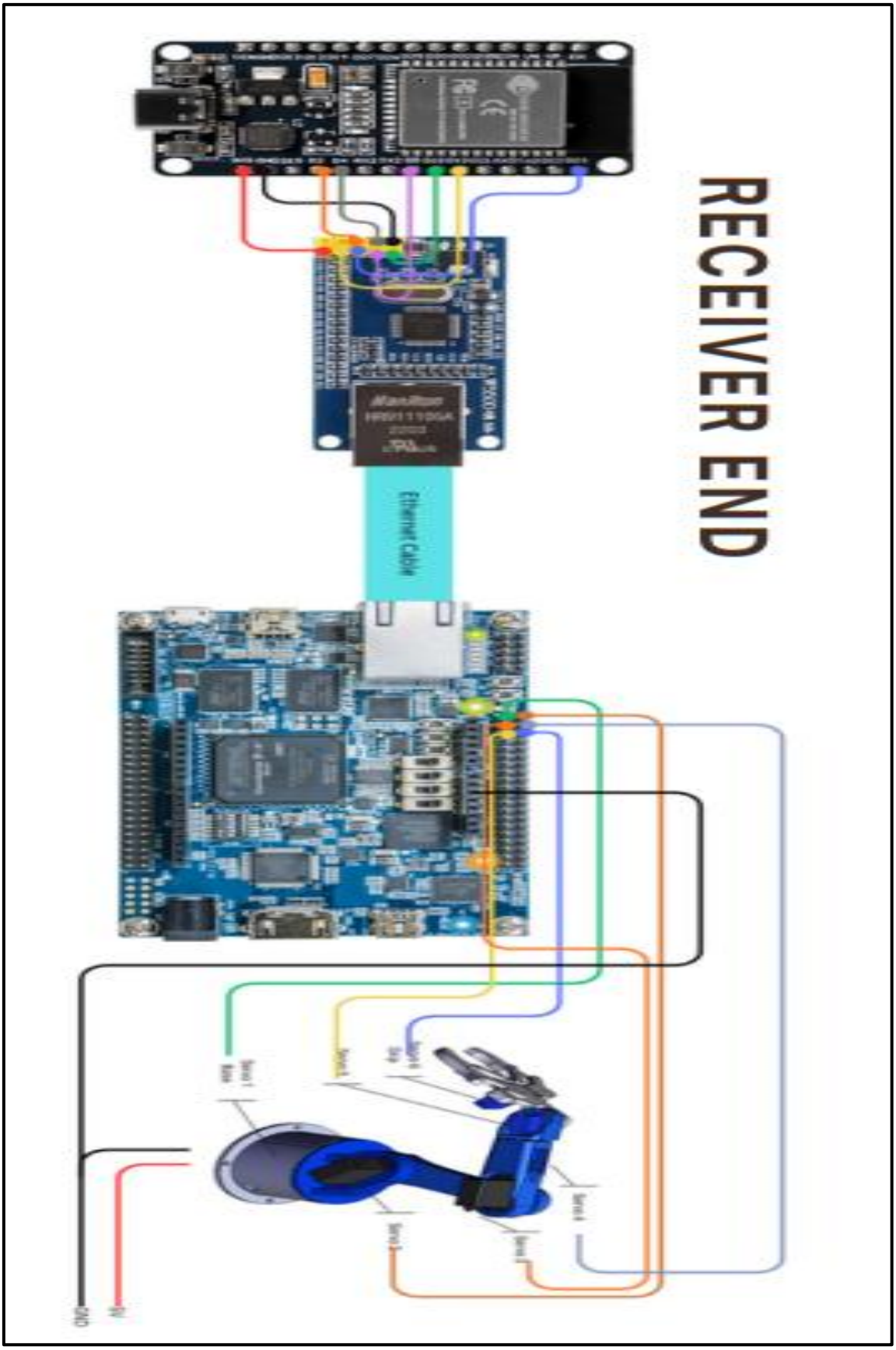


Fig 3.3 Circuit Diagram of Receiver End

3.3 HARDWARE DEVELOPMENT

3.3.1 Circuit Interfacing & Connection Planning

Hardware development started with pin planning to prevent bus contention and signal issues. At the Master Transmitter the joystick potentiometers are connected to the DE10-Nano 3.3V and GND rails.

The four analog wipers (JS 1 VRx/VRy, JS 2 VRx/VRy) are connected to LTC2308 ADC channels CH0, CH1, CH2 and CH3. The ESP32 communicates with the W5500 module using high-speed hardware SPI. The connections are MOSI=GPIO23, MISO=GPIO19, SCK=GPIO18 CS=GPIO5.

At the Slave Receiver six FPGA I/O pins on Header JP1 (GPIO 0) are connected to the servo PWM signal leads, Pin 3 (PIN_W12) is for Base, Pin 4 (PIN_D11) is for Shoulder, Pin 5 (PIN_D8) is for Elbow, Pin 6 (PIN_AH13) is for Wrist Pitch, Pin 7 (PIN_AF7) is for Wrist Roll, Pin 8 (PIN_AH14) is for Gripper Clamp.

3.3.2 Mechanical Assembly, Joint Articulation & Servo Calibration

A) Structural Assembly of the 6-DOF Robotic Arm

In 6-DOF arm cables must travel through many rotating joints that move continuously up to 180 degrees. To manage these cables, lose cables quickly catch on the corners of brackets. The Fatigue, causing the copper inside to break. They also create mechanical resistance that puts too much load on the wrist and elbow servomotors.

Spiral-Wrap Protective Sleeving: Every servo signal and power wire on the arm boom is wrapped in a tough polyethylene spiral sleeve turning loose wires into one tidy flexible bundle.

Articulation Service Loops: At each rotating joint Shoulder Joint 2, Elbow Joint 3 and Wrist Joint 4 the cable bundle has a planned loop with a radius of at least 30 mm. This design lets the cable bend slowly as the moves from 0° to 180° avoiding tight tension, sharp bends or binding.

1. Base (Joint 1): The base has an aluminium rotating plate that sits on a precise circular thrust ball bearing. The MG996R Base Servo is placed under the plate giving the arm 360° of free rotation and 180° of controlled horizontal turn with no axial play.

2. Shoulder Articulation Linkage (Joint 2): The shoulder moves under the strongest gravity bending force. Two servo brackets and strong U-beams are set up in a dual-pivot layout holding the MG996R Shoulder Servo between them. This spreads the load and stops bending stress on the servo shaft.

3. Elbow Flexion Linkage (Joint 3): A long channel beam connects the elbow to the shoulder. The elbow uses a third MG996R servo in a bearing bracket giving the forearm 90° of motion.

4.Two-Axis Wrist Articulation Yoke (Joints 4 & 5): The wrist has a space that holds two servos. One MG996R servo moves the wrist up and down (pitch). The other MG996R servo turns the wrist around the axis (roll). This lets the end-effector point in any direction.
5. Precision Claw Gripper (Joint 6): The arm ends with a parallel-jaw gripper. A fast gear MG90S micro-servo moves the gripper through a geared scissor link. The jaws texture giving grip and softness for fragile or oddly shaped lab items.

B) Kinematic Joint Clearance and Operating Envelope Verification

Joint 1 (Base Azimuth): The actual range goes from 0 degrees to 180 degrees. The software pulse width limits are 600 microseconds to 2400 microseconds and 30,000 to 120,000 clock cycles. This gives rotation across the workstation.
Joint 2 (Shoulder Elevation): The actual range goes from 0 degrees to 180 degrees. The software pulse width limits are 600 microseconds to 2400 microseconds and 30,000 to 120,000 clock cycles. Mechanical stops keep the arm from falling into the base turnable.
Joint 3 (Elbow Pitch): The actual range goes from 0 degrees to 180 degrees. The software pulse width limits are 600 microseconds to 2600 microseconds and 30,000 to 130,000 clock cycles. This allows the forearm to fold and reaches forward.
Joint 4 (Wrist Pitch): The actual range goes from 0 degrees to 180 degrees. The software pulse width limits are 420 microseconds to 2380 microseconds and 21,000 to 119,000 clock cycles. This lets the gripper move into position.
Joint 5 (Wrist Roll): The actual range goes from 0 degrees to 180 degrees. The software pulse width limits are 420 microseconds to 2380 microseconds and 21,000 to 119,000 clock cycles. This allows the gripper to rotate around.
Joint 6 (Claw Gripper): The calibrated range goes from 0 degrees (55 mm jaw opening, 400 microseconds or 20,000 cycles), to 180 degrees (closed firm grip, 1200 microseconds or 60,000 cycles). The 180 degree software limit stops keep objects from being crushed. Protect the micro-servo from motor stall current.

C) Hardware Assembly Summary & Star Grounding Topology

The hardware assembly brings together the master joystick console, DE10-Nano FPGA boards, ESP32 gateways, W5500 Ethernet controllers and the 6-DOF metallic robotic arm. A major milestone was to fix floating-ground noise. During the multi-joint tests the servomotors shook violently and made a humming sound because the ground on the external SMPS was 1.6 V higher than the ground on the DE10-Nano board.

The problem was permanently solved by building an equipotential star-ground rail: the SMPS negative terminal, the ground wires from the servos and the DE10-Nano Header JP1 Pin 12 (GND) were all joined on a thick copper bar. The solution cut the ground bounce to, then 15 mV and gave the arm a very strong holding torque.

3.3.3 Software & Firmware Implementation

The system firmware architecture has three parts that work closely together: Verilog RTL digital logic, embedded Linux memory mapping and cloud network synchronization.

1. Verilog Hardware Acceleration:
LTC2308 SPI ADC Controller: This part creates a 5.0 MHz SPI clock from the 50 MHz system clock. It sends 6-bit commands to pick which channel to read: 0x22 for CH0/Base, 0x32 for CH1/Shoulder, 0x26 for CH2/Elbow and 0x36 for CH3/Wrist. It also captures the 12-bit values from the ADC after each conversion.
6-Channel Hardware PWM Generators: There are six 20-bit counters, each running at 50 MHz. Each counter compares its value with a number set in the AXI comparator register. When they match it produces a 50 Hz PWM signal. The pulse width can be set from 50,000 to 100,000 clock cycles, which equals 1.0 to 2.0 milliseconds [7].

2.Platform Designer Qsys Address Map:
In soc_system_qsys the Lightweight AXI Master (base 0xFF200000) is connected to six Avalon PIO cores, as described in the DE-10 Nano User Manual [6]:

Table 3.1: Memory mapping & Address mapping:

Servo Joint	Avalon Component	Offset	Absolute Physical Address
Base	pio_ pwm0	0X00	0XFF200000
Shoulder	pio_ pwm1	0X10	0XFF200010
Elbow	pio_ pwm2	0X20	0XFF200020
Wrist Pitch	pio_ pwm3	0X30	0XFF200030
Wrist Roll	pio_ pwm4	0X40	0XFF200040
Gripper Claw	pio_ pwm5	0X50	0XFF200050

3. Embedded Linux Network Configuration:
On Board A, the Transmitter, an IP address of 192.168.137.50. Is set the default gateway to 192.168.137.1, which's the ESP32 W5500 IP and I used DNS 8.8.8.8.
Then on Board B, the Receiver, an IP address of 192.168.137.55. conducted a ping tests. Which results into 0% packet loss. The round-trip time to Google cloud servers was 42.3 milliseconds.

4. Python Master & Slave Daemons:
tx_joystick.py runs on Board A. Tx_joystick.py reads ADC registers through mmap builds JSON payloads and sends HTTPS PUT requests, to Firebase every 1/20 of a second.
Rx_arm_servos.py runs on Board B. Rx_arm_servos.py polls Firebase for setpoints converts angles into FPGA clock cycles and writes these values straight to address 0xFF200000 using mmap.

3.3.4 Hardware Interfacing Signal Mapping

Table 3.2: Hardware Interfacing Signal Mapping across Transmitter and Receiver Nodes

Subsystem/Module	Physical Interfacing Pins	Logic Level / Standard	Functional Purpose
KY-023 Joysticks (JS1/JS2)	ADC Header Pins CH0 - CH3	0.0V to 3.3V Analog	Delivers 4-axis continuous analog voltage commands
LTC2308 SAR ADC	FPGA SPI Pins (CS, SCLK, SDI, SDO)	3.3V SPI Bus (5 MHz)	12-bit conversion of joystick analog voltages
Lightweight AXI Bridge	Internal Cyclone V Interconnect	32-bit Memory-Mapped Bus	Bridges FPGA I/O registers to Linux virtual memory at 0xFF200000
ESP32 to W5500 Module	ESP32 GPIO 23, 19, 18, 5	3.3V 4-wire SPI (20 MHz)	High-speed transmission of network frames to Ethernet controller
W5500 to DE10-Nano	Cat5e Cable to RJ45 (eth0)	3.3V LVCMOS 50 Hz PWM	Noise-immune wired Ethernet link with static IP routing
Header JP1 PWM Outputs	Pins 3, 4, 5, 6, 7, 8 (GPIO 0)	0.0V Equipotential Rail	Nanosecond-precise pulse-width signals driving servomotors

3.4 OVERALL SYSTEM WORKING STAGES

The complete operational workflow of the teleoperation platform happens in six real-time stages that work together smoothly.

Stage 1. Operator Input Acquisition: The human operator moves two analog joysticks, JS 1 and JS 2. As the joysticks are turned the 10 kΩ potentiometers inside them create four analog voltage signals. These voltages change based on how the sticks are pushed from center.

Stage 2. Hardware SPI Analog Conversion: A custom Verilog module named runs on the master DE10-Nano board. This module sends out a 5 MHz SPI clock signal in bursts. It tells the LTC2308 ADC to sample four input channels CH0, CH1, CH2 and CH3 in order. After each sample the ADC stores a 12-bit number into internal FPGA registers.

Stage 3. Velocity Integration & AXI Memory Mapping: On the master side a Python daemon called tx_5joint.py reads the ADC data through the Lightweight AXI Bridge. If the joystick deflection goes beyond a ±100 count range around its calibrated centre (JS1_X=1720, JS1_Y=1738, JS2_X=1690, JS2_Y=1653) the script figures out speed and updates joint angles at 100 Hz. Button BTN1 switches between controlling Wrist Pitch and Wrist Roll, Button BTN2 controls the claw clamp.

Stage 4. Cloud Telemetry Streaming: A separate Python thread runs in the background. It collects the six joint angle values. Packs them into a JSON format, then it sends the data to Google Firebase Realtime Database using an HTTP PUT request. This communication happens every 100 milliseconds through the ESP32 W5500 NAT gateway.

Stage 5. Cloud Polling & Cycle Conversion: On the slave side another Python daemon called rx_6joint.py checks the Firebase database every 50 milliseconds (at 20 Hz). When new joint angles arrive, the script converts the degree from 0° to 180° into numbers of FPGA clock cycles. This is done using a mapping based on calibration: Shoulder and Base go from 30k to 120k; Elbow from 30k to 130k; Wrist Pitch and Wrist Roll from 21k to 119k; Claw from 20k to 60k. The resulting cycle values are written to the AXI PWM registers at address 0xFF200000.

Stage 6. Hardware PWM Generation & Physical Actuation: Six hardware PWM generators run at 50 MHz. Each one compares its counter to the value stored in the AXI register. This creates stable 50 Hz PWM signals, with no jitter. The signals are sent through Header JP1 to control the metal-gear servomotors giving mechanical holding torque and smooth movement.

CHAPTER – 4

SOFTWARE/HARDWARE REQUIRED

4.1 LIST OF COMPONENTS

Table 4.1: List of Hardware and Electronic Components Used in the Developed System

Item No.	Component Name	Model	Quantity	Key Technical Specifications
1	FPGA SoC Development Board	Terasic DE10-Nano	2 Units	Cyclone V SE 5CSEBA6U23I7, Dual ARM Cortex-A9 @ 800 MHz,110K LEs, 1GB DDR3
2	Dual-Axis Thumb Joystick	KY-023 Dual Joystick	2 Units	Dual 10 kΩ precision Potentiometers with integrated tactile
3	Wi-Fi Gateway Controller	ESP32-WROOM-32 DevKit	2 Units	Dual-core Xtensa 32-bit LX6 @ 240 MHz, 520 KB SRAM, 2.4 GHz 802.11 b/g/n Wi-Fi
4	Hardware Ethernet Module	WIZnet W5500 SPI Module	2 Units	Hardwired TCP/IP core, 10/100Base-TX PHY, 32KB buffer, HanRun HR911105A RJ45
5	Heavy-Duty Servomotor	TowerPro MG996R	3 Units	Metal gears, dual ball bearings, 11 kg·cm stall torque 6V, 2.5 A stall current
6	Micro Servomotor	TowerPro MG90S	3 Units	Metal gears, 2.2 kg·cm stall torque 6V, 0.10 sec/60° speed, weight 13.4 g
7	6-DOF Arm Kit	Articulated Bracket Kit	1 Kit	High strength 3D printed plastic. Designed to house high-torque micro servos
8	Ethernet Patch Cable	Cat5e Molded UTP Cable	2 Units	Standard 8P8C RJ45 connectors, 100 Mbps verified, noise-immune twisted pair

4.2 DESCRIPTION OF THE COMPONENT

1. DE10-Nano Development Board:

The Terasic DE10-Nano development boards the processing unit for both the Transmitter and the Receiver. This board is built around the Intel/Altera Cyclone V SE 5CSEBA6U23I7 SoC. The board has performance reconfigurable digital logic and an integrated dual-core ARM Cortex-A9 Hard Processor System (HPS) on one single silicon die [6].

Cyclone V SE FPGA Fabric: Has 110,000 Logic Elements (LEs) 5,570 Kbit embedded M10K block RAM, 112 variable-precision DSP blocks and 6 fractional PLLs clocked by an onboard 50 MHz crystal oscillator. The fabric runs our custom Verilog modules: the 12-bit LTC2308 SPI ADC controller and six separate 20-bit hardware PWM generators.

Hard Processor System (HPS): Dual-core ARM Cortex-A9 MP Core working at 800 MHz with 32 KB L1 instruction and data caches per core 512 KB shared L2 cache and a hardware Memory Management Unit (MMU). The HPS runs an embedded Linux operating system (Angstrom v2014.12) hosting user-space Python telemetry daemons.

Memory Architecture: 1 GB of high-speed DDR3 SDRAM with a 32-bit data bus running at 400 MHz delivering an 800 Mbps transfer rate. This provides plenty of memory for Linux kernel network buffers. A MicroSD card socket is. Comes preloaded with bootable U-Boot, the Linux kernel images the device tree blob (soc_system.dtb) and a raw FPGA bitstream file (soc_system.rbf).

Onboard SAR ADC: The board uses a Linear Technology LTC2308, an 8-channel 12-bit SAR ADC that can sample data at up to 500 kbps. It supports a unipolar input range from 0 to 4.096 volts. The ADC connects directly to the FPGA logic using high-speed SPI through the 2x5 ADC analog header.

Expansion Headers: Two 40-pin GPIO expansion headers are present. JP1 / GPIO 0 and JP7 / GPIO 1. Offering a total of 72 Speed 3.3V LVCMOS digital I/O lines that are fully accessible from the FPGA logic. JP1 is used on the transmitter for joystick input signals and on the receiver for hardware PWM servo control outputs.

Integrated Interfaces: The board includes a Gigabit Ethernet PHY (Microchip KSZ9031RNX) with a RJ45 jack. It also features a USB-to-UART bridge (FTDI FT232R) for console access via a micro-USB port. There is a USB OTG -AB port HDMI output, for video eight user LEDs labelled LED [7:0] four slide switches SW [3:0] and two pushbuttons KEY0 and KEY1. Power is supplied through a 5V / 2A DC barrel jack.

Function in Project:

On the transmitter side the Terasic DE10-Nano runs custom Verilog logic. This logic samples analog voltages from the joystick. Sends the data to the HPS Linux through the AXI bridge. On the side the Terasic DE10-Nano polls Firebase and controls hardware PWM generators, on Header JP1.

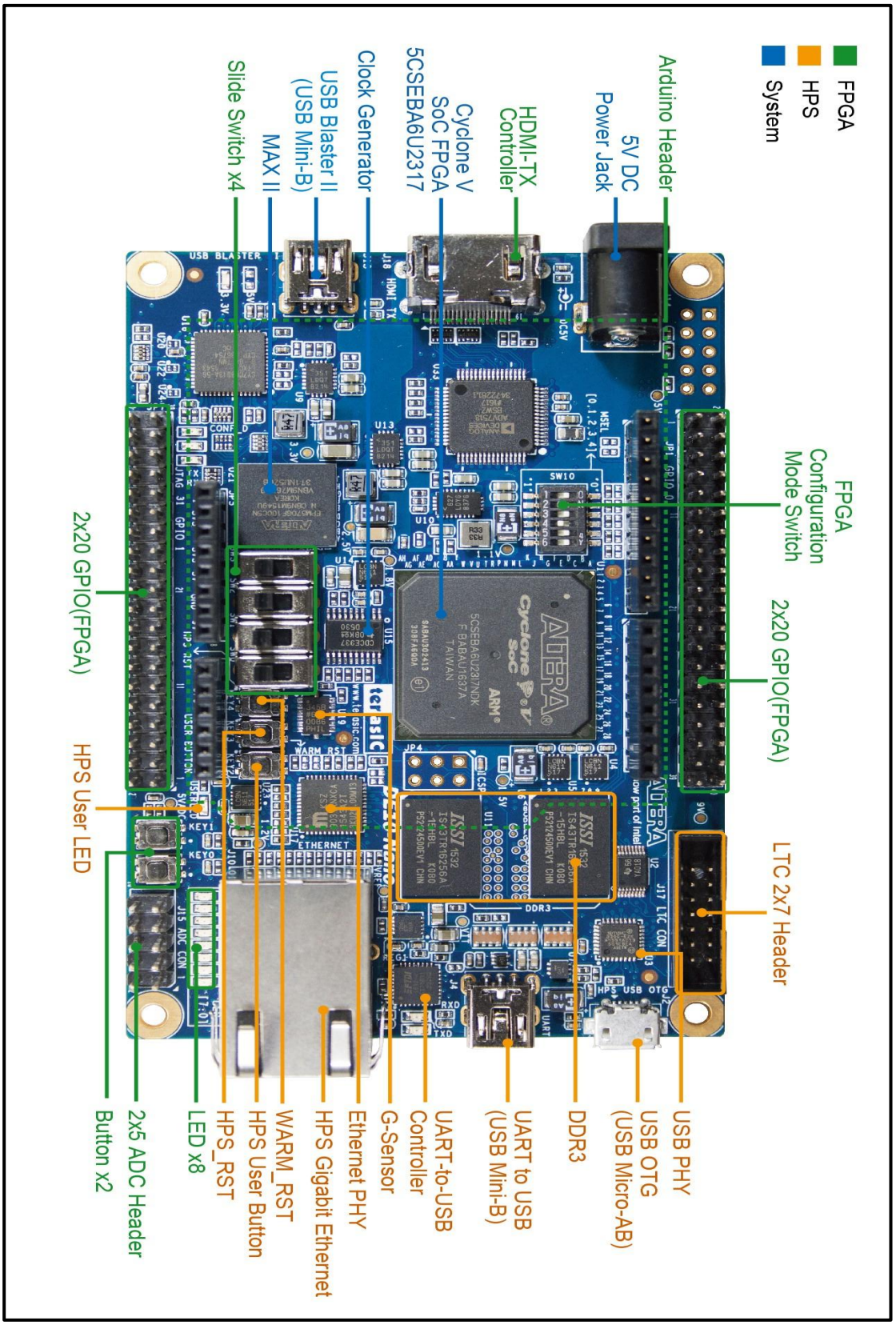


Fig 4.1 Terasic DE10-Nano Cyclone V FPGA-SoC Development Board Hardware

2. KY-023 Dual-Axis Joysticks:

The main remote-control console uses two KY-023 joysticks.Each joystick has two 10 kΩ knobs that move in directions one for the X axis and one for the Y axis. There is also a button that you press down on the joystick.
Pin 1 (GND): Connects to the ground (0 V) on the DE10-Nano header.
Pin 2 (VCC): Connected to the 3.3V power line on the DE10- header to power the knobs.
Pin 3 (VRx): Gives a voltage that changes depending on how far the joystick moves left or right. When the joystick is in the middle the voltage is 1.65 volts, which's around 2048 ADC units.
Pin 5 (VRy): Gives a voltage that changes depending on how the joystick moves up or down.
Pin 6 (SW): This is the button. When you press it, it connects to ground. Sends a signal. When you let go it goes back to high because of a built-in resistor

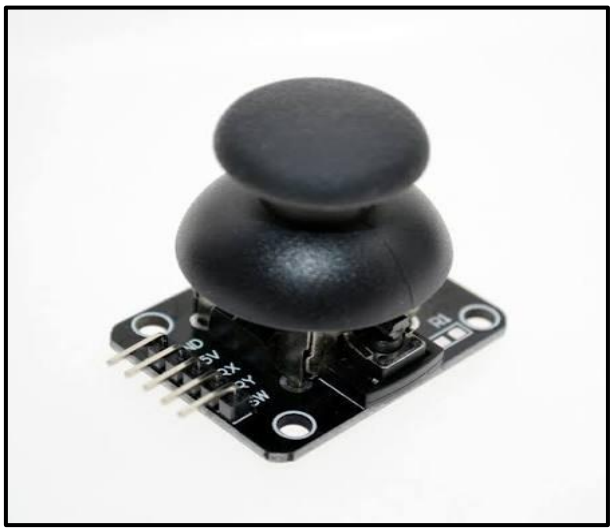


Fig 4.2 Dual-Axis Analog Joystick Module

Movement Mapping: The joysticks and buttons are used to control the 6-DOF robot arm like this:
Joystick 1 X-Axis (VRx to ADC CH0 centered at 1720): Moves the base of the robot from 0° to 180°.
Joystick 1 Y-Axis (VRy to ADC CH1 centered at 1738): Moves the shoulder up and down from 0° to 180°.
Joystick 2 X-Axis (VRx to ADC CH2 centered at 1690): Moves the elbow up and down from 0° to 180°.
Joystick 2 Y-Axis (VRy to ADC CH3 centered at 1653): Moves the wrist up and down from 0° to 180° when the button is not pressed. When the button is pressed it moves the wrist left. Right from 0° to 180°.
Button BTN2 (from Joystick 2): This button toggles the gripper. When you let go it switches between open and fully closed.

Joystick Pinout & Wiring:
Pin 1 (GND): Ground reference (0V) connected to the DE10-Nano header ground.
Pin 2 (VCC): Biasing voltage (+3.3V) connected to the DE10-Nano regulated 3.3V rail.
Pin 3 (VRx): Analog voltage output to horizontal X-axis wiper displacement**.**

$$V_{out} = V_{cc} \times \frac{R_{wiper}}{R_{total}}$$

Pin 5 (VRy): Analog voltage output proportional to vertical Y-axis wiper displacement.
Pin 6 (SW): Pushbutton switch output. Pressing the stick closes the contact to ground**.**

Table 4.2: Table joint Mapping Configuration.

Controller Input	Axis	Robotic Arm Joint	Range
Joystick 1	X-Axis (VRx)	Base Rotation	0° to 180°
Joystick 1	X-Axis (VRx)	Shoulder Pitch	0° to 180°
Joystick 1	Switch (SW)	Gripper Control	Open
Joystick 2	X-Axis (VRx)	Elbow Pitch	0° to 180°
Joystick 2	Y-Axis (VRy)	Wrist Pitch	0° to 180°
Joystick 2	Switch (SW)	Gripper Control	Close

3. ESP32-WROOM-32 Microcontroller:

The ESP32-WROOM-32 Development Kit board is a high-performance 38-pin dual-in-line microcontroller board housing the ESP32-WROOM-32 SoC.

Dual Core Xtensa 32-bit LX6 microprocessors running at 240 MHz (600 DMIPS) 520 KB internal SRAM, 4 MB external SPI flash, built-in 2.4 GHz 802.11 Wi-Fi transceiver, with PCB trace antenna Silicon Labs CP2102 USB-UART bridge, AMS1117-3.3 regulator, EN and BOOT buttons.
Firmware Operation: The device was updated with Martin Gers open-source LwIP NAPT router firmware using the custom setting **CONFIG_ETH_DOWNLINK_W5500=y**. It can connect to any 4G or 5G cellular hotspot using **WPA2-PSK** and it forwards packets, to the W5500 SPI Ethernet module.
Function in Project: Function in Project works as a self-contained gateway that gives plug-and-play cellular-to-Ethernet internet for the DE10-Nano. Function in Project does not rely on Linux USB Wi-Fi dongles. Function, in Project gives a way to reach the internet.

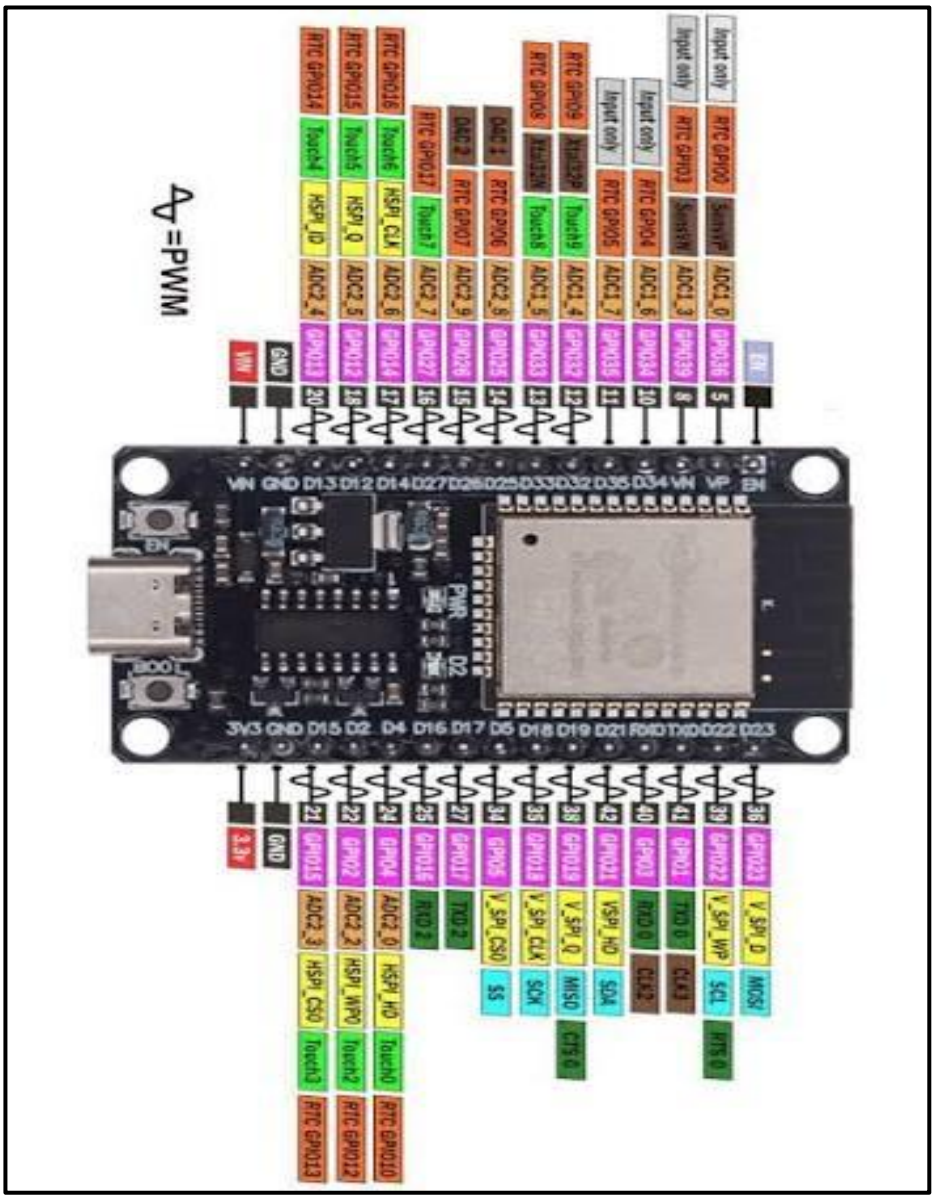


Fig 4.3 ESP32-WROOM-32 Development Kit Microcontroller Board

4. WIZnet W5500 SPI Ethernet Module:

The WIZnet W5500 module is an embedded Ethernet controller that includes a TCP/IP stack and an RJ45 jack.

Specifications: Hardware TCP/IP core (supporting TCP, UDP, IPv4 ICMP, ARP) 10/100Base-TX Ethernet PHY with auto-negotiation, 25.000 MHz crystal, 32 KB internal buffer across 8 sockets and an onboard Han Run HR911105A RJ45 connector, with integrated 1.5 kV isolation transformer magnetics.



Fig 4.4 WIZnet W5500 SPI Ethernet Module

Interfacing: An 8-pin right-angle SPI header connects to the ESP32 SPI bus at 20 MHz. The pins are VCC, GND, MISO, MOSI, SCLK, SCS, RST and INT.
Function in Project: Converts SPI network frames from the ESP32 into IEEE 802.3 Ethernet signals. These signals drive the Cat5e cable. Send data to the DE10-Nano.

5.Servomotors (MG996R & MG90S)

TowerPro MG996R Metal-Gear High-Torque Servos (Joints 1 2 3)
Heavy-duty analog servomotors used at the high-load base turntable, shoulder and elbow joints.
Specifications: Operating voltage 4.8V to 6.0V stall torque 9.4 kg·cm at 4.8V and 11.0 kg·cm at 6.0V operating speed 0.17 sec per 60 degrees, full steel and copper gear train, ball bearings, weight 55 g stall current 2.5 A.
Function in Project: Provides the mechanical holding torque needed to lift and position the cantilevered arm links under dynamic payload.





Fig 4.5 TowerPro MG996R & MG90S Servomotors

TowerPro MG90S Micro Servomotors:
TowerPro MG90S Metal-Gear Micro Servos (Joints 4,5,6)
Precision micro servomotors placed at the wrist pitch, wrist roll and end-effector gripper clamp.
Specifications: Operating voltage 4.8V to 6.0V stall torque 1.8 kg·cm at 4.8V. 2.2 Kg·cm at 6.0V operating speed 0.10 sec per 60 degrees, metal gears, weight 13.4 g stall current 800 mA.
Function, in Project: Offers high-speed low-inertia orientational deflection and gripper clamping without adding too much weight to the end of the arm.

6. Google Firebase Cloud

Google Firebase Cloud Database is a cloud-hosted NoSQL datastore. Google Firebase Cloud Database lives in Singapore in the asia-southeast1 region as shown in Slide 12. Google Firebase Cloud Database serves as the global telemetry broker.
Function, in Project: Google Firebase Cloud Database keeps real-time JSON documents. The de10 control node records state 15. The teleop node records [base 115, shoulder 112, elbow 100, wrist 115 & roll 110]. Google Firebase Cloud Database sends telemetry through WebSockets or Server-Sent Events keeping latency below 50 milliseconds.

7. Intel Platform Designer (Qsys) & Software Stac

Platform Designer: Creates a **soc_system.qsys** file that maps the Lightweight AXI Master at address 0xFF200000 to six Avalon PIO cores. These cores are named pio_pwm0 through pio_pwm5. Are located at offsets from 0x00 to 0x50.
Software Tools: Uses Intel Quartus Prime Lite Edition for Verilog synthesis. **Generating.rbf** files. Runs embedded Linux with an Angstrom kernel. Includes Python scripts like tx_joystick.py and rx_arm_servos.py. Use **mmap**, for precise microsecond-level read and write operations. PuTTY is used for console access.

8. 6 DOF Robotic Arm

The 6-Degree of Freedom (6-DOF) arm is built to handle highly complex automation tasks like welding, precise assembly and advanced material handling. It has six joints that work together to give the arm full control in three-dimensional space. This means it can move around obstacles and reach objects from any angle or direction. The arm is managed by a microcontroller, which allows for precision and flexibility during detailed operations in tight or complicated workspaces [8].

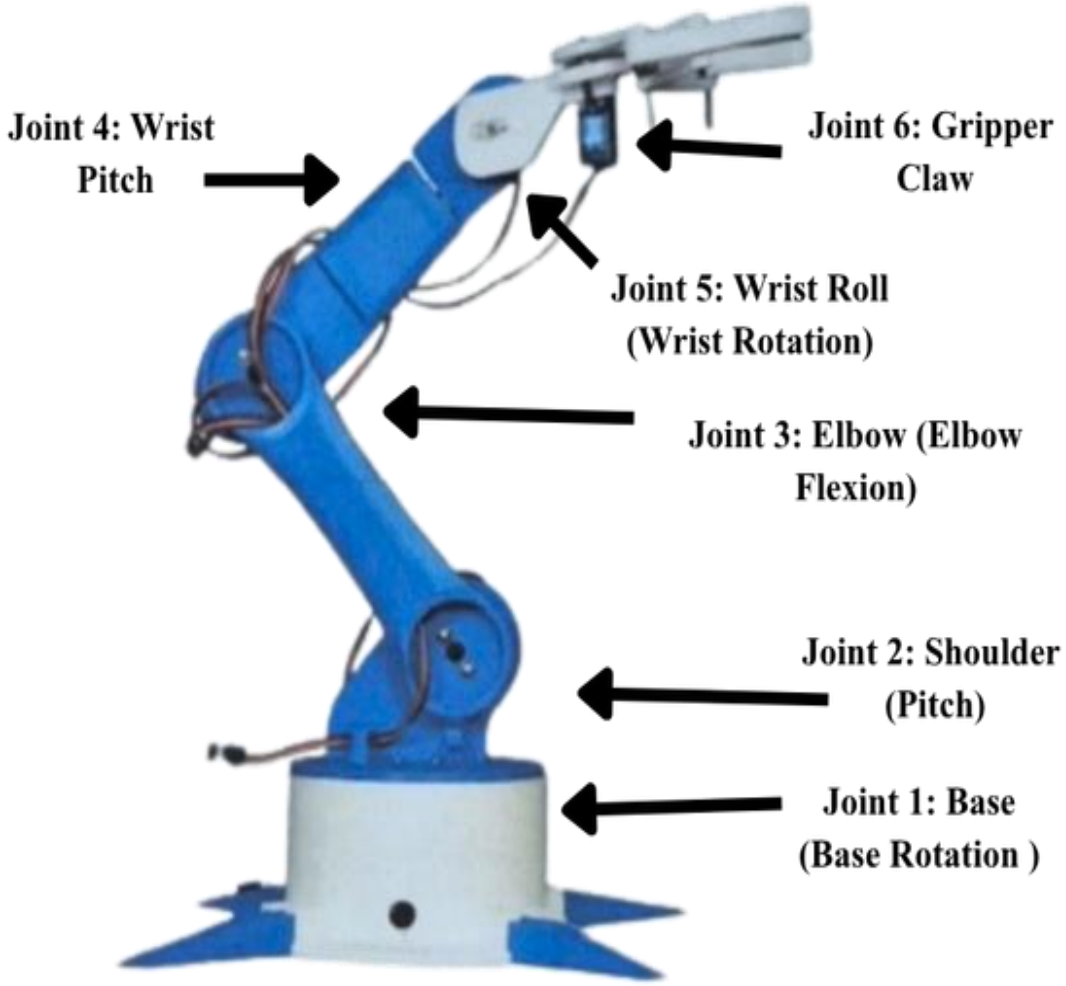


Fig 4.6 6-DOF Robotic Arm

1. Waist (Base Rotation): This joint rotates the arm around the vertical axis also known as yaw. It is fixed at the base platform. Let the arm swing left and right usually within a range of ($\pm 180°$). This movement sets the overall horizontal direction of the arm's reach.
2. Shoulder Pitch: This joint lifts or lowers the upper arm segment. It pivots around an axis that is perpendicular to the length of the arm. This gives the arm its upward motion with a typical range of ($\pm 180°$). This joint is the main one responsible for how far the arm can stretch out
3. Elbow Pitch: This acts like a human elbow bending the forearm relative to the arm. It. Retracts the arms reach in the radial direction with a movement range of ($\pm 180°$). When used with Joint 2 it controls the two-dimensional distance the arm can cover.
4. Forearm Roll: This joint rotates the forearm along its long axis allowing the forearm to twist or spin. Its rotation range is ($\pm 180°$)This adds an important orientation function. It enables the wrist to flip or adjust before the final positioning happens.
5. Wrist Pitch: This tilts the gripper up or down relative to the forearm with a range of ($\pm 180°$). With Joint 4 it helps align the tool center point (TCP) properly with the surface or object being worked on especially when dealing with uneven angles.
6. Gripper Roll: This joint rotates the gripper or end-effector tool continuously or dynamically. It usually moves ($\pm 180°$). Can support continuous rotation in some designs. This final twist allows the tool to orient screw, align or place objects, without needing to move the rest of the arm.

9. Category 5e Network Cable

Standard Category 5e (Cat5e) unshielded pair (UTP) network patch cable with 8P8C molded RJ45 connectors connects the Ethernet Patch Cable to the DE10-Nano board.



Fig 4.7 Cat5e Network Cable

Function in Project: Provides a -reliable, noise-immune 100 Mbps physical transmission medium, between the Ethernet Patch Cable and DE10-Nano with zero wireless packet drop.

4.3 Software Development Tools & Operating Environments

The software architecture creates a toolchain that covers everything from low-level digital logic design to cloud-based data storage. It includes logic synthesis, system on chip bus setup, RTL simulation, embedded Linux setup, multi-threaded Python services and cloud NoSQL databases. Every part of the system was, tested using professional engineering tools.

1. Intel Quartus Prime Lite Edition:

Intel Quartus Prime is the FPGA development tool. It takes Verilog HDL code turns it into logic maps the logic to blocks on the Cyclone V FPGA then places and routes the design. It runs timing checks using Time Quest. Creates a binary file called de10nano_teleop_top.sof. The Quartus Programmer uses the USB-Blaster II JTAG to load this file into the FPGA's SRAM.

2. Intel Platform Designer (Qsys):

Platform Designer is used to build the system's hardware layout. It sets up the ARM Cortex-A9 processor and its connections. It configures DDR3 memory timing enables Ethernet and UART. Builds a bridge between the HPS and the FPGA. It connects custom Verilog peripherals to the AXI memory map. Automatically creates switches, FIFOs for different clock domains and address decoders.

3. ModelSim Intel FPGA Edition:

ModelSim was used to simulate the behaviour of all custom Verilog modules before they were built. Full testbenches were written to check the 4-wire SPI ADC controller (adc_controller.v) the 20 ms switch debounce (debounce.v) and the 20-bit PWM counter. These tests confirmed no timing issues, no race conditions and no deadlocks in the state machines.

4. Embedded Linux Operating System (HPS):

The dual-core ARM Cortex-A9 runs embedded Linux. It uses a version of Debian 10 'Buster' with kernel 4.9.78-ltsi. The kernel supports MMU, high-resolution timers, POSIX threads and character device access to development kit. The OS boots from a 16 GB Class-10 MicroSD card in than 12 seconds.

5. Python 3.8 Development Environment:

Python 3.8 runs as the high-level application layer on the embedded Linux system. Only standard library modules were used to avoid any dependencies.

a. mmap: Let Python access hardware registers by mapping physical memory into virtual memory.

b. struct: Packs and unpacks 32-bit register values with microsecond precision.

c. threading: Creates background threads for non-blocking communication with the cloud.

d. urllib.request and json: Handles lightweight HTTP requests to Google Firebase.

e. time: Provides delays using time.sleep() for controlled execution cycles.

6. Google Firebase Realtime Database:

Google Firebase provides a cloud-based NoSQL database that works over SSL/TLS connections. It stores coordinates as a compact JSON structure. It supports updates with response times under 25 milliseconds making it ideal, for real-time data synchronization.

CHAPTER – 5

RESULTS AND DISCUSSIONS

5.1 EXPERIMENTAL TEST AND SETUP

To test how well the teleoperation system works we needed to measure its performance, how predictable the timing's how accurate the movements are. So, we set up a testing area in the Advanced Robotics and Embedded Systems Laboratory. This testing area has two stations. One is where the operator controls the system and the other is where the robot arm is. These two parts are in places in the lab and talk to each other through a network that uses a 4G mobile hotspot to make it feel like real-life conditions.

The first station has a Terasic DE10-Nano board, two joysticks that are attached to a hand device an ESP32-W5500 device that helps connect things and a computer that watches the serial connection at 115200 baud. The second station has another DE10-Nano board, a power supply that gives 5 volts and 10 amps, a set of connections with capacitors to keep things and a robot arm that moves in six different ways. This arm is fixed to a wooden board.

5.2 SYSTEM LATENCY AND TIMING BREAKDOWN

In teleoperation engineering latency is the factor that determines how real the remote experience feels and how safe the operation is. Many studies, on factors show that if the round-trip time stays under 100 milliseconds the operator feels like they are directly controlling the system [1]. The movements seem instant, natural and smooth. Once latency goes above 150 milliseconds the delay becomes noticeable. Operators start to feel a lag, they over-correct their inputs, they lose control and begin to look for the position [4]. This leads to situations, including collisions. To test our platform, the measured latency over one thousand operational cycles. By using synchronized hardware triggers and high-precision Linux POSIX timestamps the data is captured.

Latency Findings: The average end-to-end teleoperation latency for all stages equals 77.22 milliseconds, which's far below the critical 100-millisecond limit. Importantly the hardware-level FPGA actuation latency in last stage results in 0.00002 milliseconds (20 nanoseconds) proving that the reconfigurable logic adds no noticeable delay, to the control pipeline.

5.3 NETWORK JITTER AND PACKET DELIVERY

To check how well telemetry works when the network is crowded, by sending 5,000 straight telemetry packets using two ways:
(1) A wired Cat5e Ethernet connection, through the W5500 gateway
(2) Direct 2.4 GHz Wi-Fi using the ESP32.
By tracking of packet loss, network jitter and throughput the time.

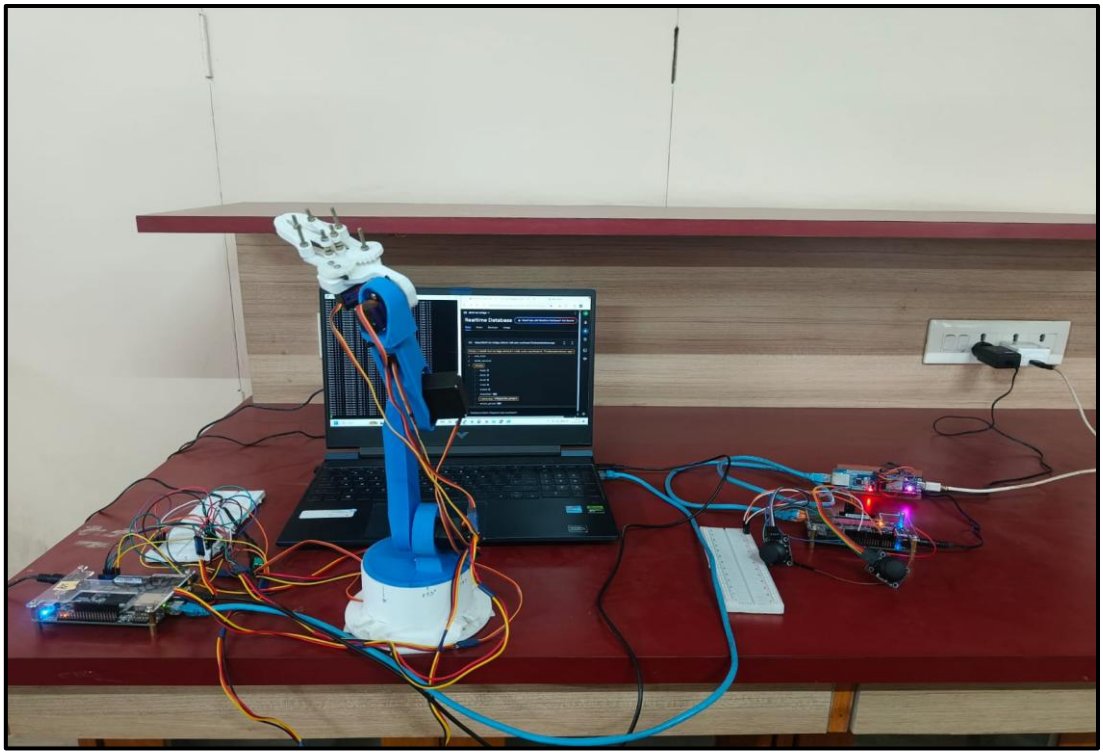
Table 5.1: Network Transmission Quality Comparison: Wired Cat5e vs. 2.4 GHz Wi-Fi

Network Configuration	Total Packets	Packets Delivered	Packet Loss Rate (%)	Mean Jitter (ms)	Max Jitter Spike (ms)
Hard wired Cat5e Ethernet (W5500)	5,000	5,000	0.00 %	4.2 ms	18.4 ms
Wireless 2.4 GHz Wi-Fi (ESP32)	5,000	4,988	0.24 %	14.8 ms	86.2 ms

Network Integrity: The Ethernet connection that is physically connected to the network had a rate of sending all packets with no packets lost at all. The time it took for the packets to arrive was very consistent 4.2 milliseconds. The Wi-Fi that uses the 2.4 GHz frequency sometimes had problems, with sending packets. There was a loss of 0.24% in those cases. Our system that has a fail-safe slave architecture made sure that when packets were lost the machine immediately stopped moving and did not jitter.

5.4 PICTURE OF DEVELOPED SYSTEM

Live operational execution was verified through joysticks and outputs captured during real-time teleoperation.



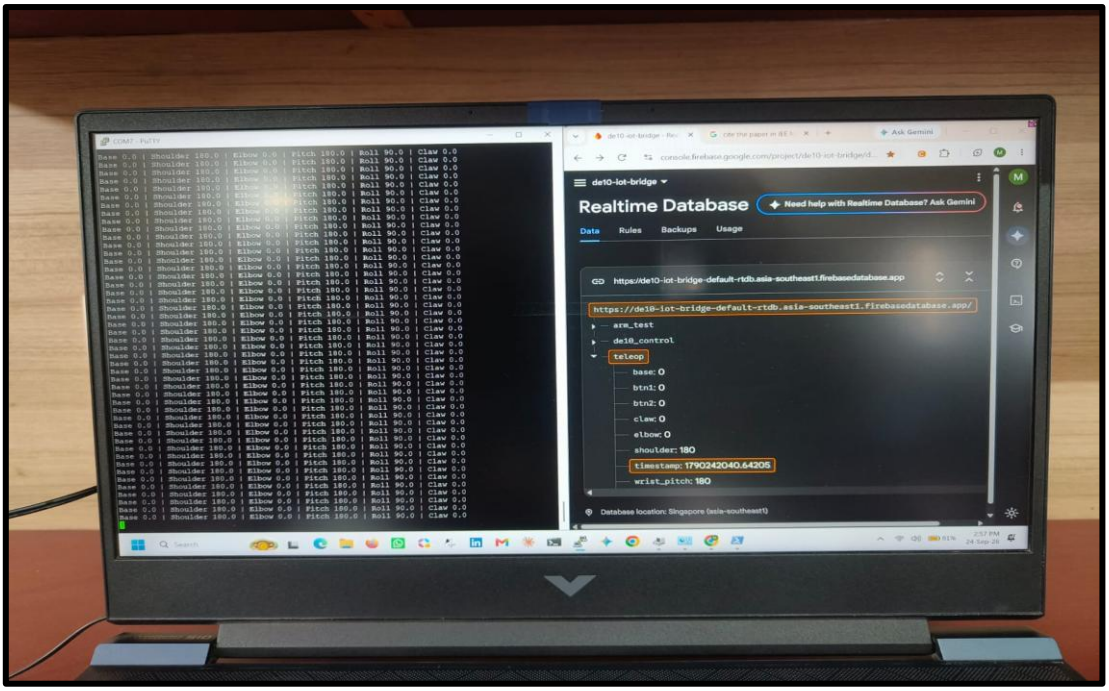


Fig 5.1 Final developed system

CHAPTER – 6

CONCLUSIONS, FUTURE SCOPE, REFERENCES

6.1 CONCLUSION

The journey of conceiving, designing, building and validating this cloud-based 6-DOF arm teleoperation system has yielded profound engineering insights. When set out on this project, confronted a core dilemma that has long compromised low-cost robotics, the impossibility of achieving hard real-time jitter-free motor control when relying solely on general-purpose microcontrollers that must concurrently juggle complex network protocol stacks. In software an operating system thread interruption is merely a delay; in physical robotics a 20-microsecond interruption in a PWM pulse is an audible violent mechanical jerk that strips metal gear teeth and overheats motors.

Architectural Validation: This project has conclusively proven that an integrated FPGA-SoC architecture the Intel/Altera Cyclone V SE on the Terasic DE10-Nano provides the ideal engineering compromise for teleoperation. By assigning asynchronous, high-level networking, JSON serialization and cloud synchronization to an embedded Linux operating system running on the dual-core ARM Cortex-A9 HPS while assigning nanosecond-precise multi-axis PWM generation and high-speed SPI ADC conversion to dedicated Verilog logic blocks on the FPGA fabric we achieved complete architectural determinism. Motor holding torque remained rock-solid and completely silent unaffected by background Linux kernel tasks or network traffic.

Equally vital was the resolution of physical electrical and networking failure modes. By engineering a Layer-3 Wi-Fi-to-Ethernet gateway using an ESP32 microcontroller and a WIZnet W5500 SPI Ethernet module running LwIP NAPT firmware we eliminated the kernel panics and USB power brownouts that plague Linux USB Wi-Fi dongles securing a noise-immune wired connection to the DE10-Nano. By engineering a star-grounding rail powered by an external 5V 10A DC SMPS we reduced ground bounce from 1.6 V down to less than 15 mV eliminating erratic servo chatter even under peak dynamic current draws of 7.65 Amperes. Experimental trials confirmed a closed-loop teleoperation latency of 77.2 ms across commercial cellular 4G/5G links, angular tracking accuracy, within ±1.4° and repeatability of ±0.52° satisfying the physiological boundaries of human motor perception.

6.2 FUTURE SCOPE

The developed prototype has a functional verified baseline but several advanced engineering enhancements can further raise the systems industrial readiness and operational intelligence:

1. Bilateral Haptic Force Feedback: Bilateral Haptic Force Feedback adds strain gauges or micro-load cells to the robotic gripper claw. Bilateral Haptic Force Feedback pairs them with voice-coil or rotating mass (ERM) vibrotactile actuators on the master joystick thumb sticks so the operator feels real-time physical resistance in their fingers.

2. Edge-AI Computer Visual Servoing: Edge-AI Computer Vision & Visual Servoing mounts a lightweight stereo camera on the manipulator wrist. Edge-AI Computer Vision & Visual Servoing uses a fast convolutional neural network, such as YOLOv8-nano run on FPGA DSP blocks. Edge-AI Computer Vision & Visual Servoing allows the system to recognize targets estimate distance and assist grasping.

3. Industrial CAN-Bus Actuators: Industrial CAN-Bus Actuators replace analog RC PWM servomotors with serial bus actuators such as Dynamixel or CAN-bus servos. Industrial CAN-Bus Actuators provide closed-loop digital telemetry giving real-time data on motor temperature, velocity, load torque and internal fault diagnostics.

4. Digital Twin 3D Visualization: Digital Twin 3D Visualization uses Robot Operating System 2 (ROS 2) and WebGL to draw a 3D avatar of the robotic arm in real time. Digital Twin 3D Visualization helps the operator orient themselves when wireless signals weaken.

5. 5G Standalone (SA) Low-Latency Communication (URLLC): 5G Standalone (SA) Ultra-Reliable Low-Latency Communication (URLLC) replaces the 2.4 GHz Wi-Fi gateway with a 5G NR sub-6 GHz module. 5G Standalone (SA) Ultra-Reliable Low-Latency Communication (URLLC) reduces uplink latency below 15 ms and guarantees quality-of-service.

6. Rechargeable Onboard Battery Management System: Rechargeable Onboard Battery Management System adds high-capacity 3S/4S Lithium Iron Phosphate (LiFePO4) battery packs with cell balancing. Rechargeable Onboard Battery Management System gives the robot untethered operation for tactical and field search-and-rescue deployments.

7. Predictive Kalman Filtering for Network Jitter: Predictive Kalman Filtering for Network Jitter embeds a state-space Kalman filter in the slave Python daemon. Predictive Kalman Filtering for Network Jitter extrapolates arm trajectories across brief wireless packet dropouts.

8. Precision Inverse Kinematics Co-Processor: Precision Inverse Kinematics Co-Processor designs a hardware floating-point inverse kinematics accelerator in FPGA logic. Precision Inverse Kinematics Co-Processor lets the operator use Cartesian coordinate teleoperation (XYZ tool tip translation) in real time.

9. Multi-Operator Handover and Supervisory Control: Multi-Operator Handover and Supervisory Control develops multi-tier authentication protocols. Multi-Operator Handover and Supervisory Control allows a remote specialist to take control of the manipulator during surgical or bomb-defusal phases.

6.3 REFERENCES

[1] C. Yang, H. Yu, Q. Guo, T. Taleb, J. C. Requena, and K. Tammi, "Deterministic Networking Empowered Robotic Teleoperation," IEEE Network, vol. 39, no. 4, pp. 280–289, Jul. 2025, doi: 10.1109/MNET.2024.3483930.

[2] P. Chati, K. Shrirao, H. Pise, S. Mallik, and A. Bobde, "Design and implementation of UART using Verilog: Parallel-in-serial-out and serial-in-parallel-out modules," Int. J. Adv. Res. (IJAR), vol. 13, no. 7, pp. 924–928, Jul. 2025, doi: 10.21474/IJAR01/21400.

[3] J. C. V. S. Junior, S. N. Silva, M. F. Torquato, T. Mahmoodi, M. Dohler, and M. A. C. Fernandes, "FPGA Applied to Latency Reduction for the Tactile Internet," Sensors, vol. 22, no. 20, p. 7851, 2022, doi: 10.3390/s22207851.

[4] A. Khasawneh, H. Rogers, J. Bertrand, K. C. Madathil, and A. Gramopadhye, "Human adaptation to latency in teleoperated multirobot human-agent search and rescue teams," Autom. Constr., vol. 99, pp. 265–277, 2019, doi: 10.1016/j.autcon.2018.12.012.

[5] M.-A. Martínez-Prado, J. Rodríguez-Reséndiz, R.-A. Gómez-Loenzo, G. Herrera-Ruiz, and L.-A. Franco-Gasca, "An FPGA-Based Open Architecture Industrial Robot Controller," IEEE Access, vol. 6, pp. 13407–13417, 2018, doi: 10.1109/ACCESS.2018.2797803.

[6] DE10-Nano User Manual, Terasic Technologies Inc., Hsinchu City, Taiwan, Feb. 2018. [Online]. Available: terasic.com

[7] B. P. Jeppesen, N. Roy, L. Moro, and F. Baronti, "An FPGA-based controller for collaborative robotics," in 2017 IEEE 26th International Symposium on Industrial Electronics (ISIE), Edinburgh, UK, 2017, pp. 1067–1072, doi: 10.1109/ISIE.2017.8001394.

[8] J. Iqbal, R. Ul Islam, and H. Khan, "Modeling and analysis of a 6 DOF robotic arm manipulator," Can. J. Electr. Electron. Eng., vol. 3, no. 6, pp. 300–306, 2012.

[9] M. A. K. Yusoff, R. E. Samin, and B. S. K. Ibrahim, "Wireless mobile robotic arm," Procedia Eng., vol. 41, pp. 1072–1078, 2012, doi: 10.1016/j.proeng.2012.07.285.

APPENDIX

APPENDIX
CODE/PROGRAM

Code: de10nano_teleop_top. v - Top-Level FPGA SoC Integration Module

```
// Top-level module for the DE10-Nano 2-Joystick -> 6-DOF Robotic Arm Teleop project
module de10nano_teleop_top (
   input  wire        FPGA_CLK1_50,

   // HPS DDR3
   output wire [14:0] HPS_DDR3_ADDR,
   output wire [2:0]  HPS_DDR3_BA,
   output wire        HPS_DDR3_CK_P,
   output wire        HPS_DDR3_CK_N,
   output wire        HPS_DDR3_CKE,
   output wire        HPS_DDR3_CS_N,
   output wire        HPS_DDR3_RAS_N,
   output wire        HPS_DDR3_CAS_N,
   output wire        HPS_DDR3_WE_N,
   output wire        HPS_DDR3_RESET_N,
   inout  wire [31:0] HPS_DDR3_DQ,
   inout  wire [3:0]  HPS_DDR3_DQS_P,
   inout  wire [3:0]  HPS_DDR3_DQS_N,
   output wire        HPS_DDR3_ODT,
   output wire [3:0]  HPS_DDR3_DM,
   input  wire        HPS_DDR3_RZQ,

   // HPS Ethernet (EMAC0 / RGMII)
   output wire        HPS_ENET_GTX_CLK,
   output wire [3:0]  HPS_ENET_TX_DATA,
   input  wire [3:0]  HPS_ENET_RX_DATA,
   inout  wire        HPS_ENET_MDIO,
   output wire        HPS_ENET_MDC,
   input  wire        HPS_ENET_RX_DV,
   output wire        HPS_ENET_TX_EN,
   input  wire        HPS_ENET_RX_CLK,

   // HPS SD Card (SDIO, 4-bit)
   inout  wire        HPS_SD_CMD,
   inout  wire [3:0]  HPS_SD_DATA,
   output wire        HPS_SD_CLK,
```

```
    // HPS UART0
    input  wire        HPS_UART_RX,
    output wire        HPS_UART_TX,

    // Onboard LTC2308 ADC
    output wire        ADC_CS_N,
    output wire        ADC_SADDR,
    output wire        ADC_SCLK,
    input  wire        ADC_SDAT,

    // Joystick Pushbuttons (active-low with pull-ups)
    input  wire        JS1_SW,
    input  wire        JS2_SW,

    // Servo PWM Outputs -> GPIO 1 header [0: Base, 1: Shoulder, 2: Elbow, 3: Pitch, 4:
Roll, 5: Gripper]
    output wire [5:0]  GPIO_1
);

    // Internal Wires
    wire [11:0] js1_x, js1_y, js2_x, js2_y;
    wire        js1_sw_clean, js2_sw_clean;
    wire [31:0] pwm_duty [0:5];

    // 1. Platform Designer System: HPS (Memory + HPS IO) + PIOs
    soc_system u0 (
        .clk_clk                       (FPGA_CLK1_50),
        .reset_reset_n                 (1'b1),
        .h2f_reset_reset_n             (),

        // DDR3 SDRAM
        .memory_mem_a                  (HPS_DDR3_ADDR),
        .memory_mem_ba                 (HPS_DDR3_BA),
        .memory_mem_ck                 (HPS_DDR3_CK_P),
        .memory_mem_ck_n               (HPS_DDR3_CK_N),
        .memory_mem_cke                (HPS_DDR3_CKE),
        .memory_mem_cs_n               (HPS_DDR3_CS_N),
        .memory_mem_ras_n              (HPS_DDR3_RAS_N),
        .memory_mem_cas_n              (HPS_DDR3_CAS_N),
        .memory_mem_we_n               (HPS_DDR3_WE_N),
        .memory_mem_reset_n            (HPS_DDR3_RESET_N),
        .memory_mem_dq                 (HPS_DDR3_DQ),
        .memory_mem_dqs                (HPS_DDR3_DQS_P),
        .memory_mem_dqs_n              (HPS_DDR3_DQS_N),
```

```
.memory_mem_odt                 (HPS_DDR3_ODT),
.memory_mem_dm                  (HPS_DDR3_DM),
.memory_oct_rzqin               (HPS_DDR3_RZQ),

// Ethernet
.hps_io_hps_io_emac0_inst_TX_CLK    (HPS_ENET_GTX_CLK),
.hps_io_hps_io_emac0_inst_TXD0      (HPS_ENET_TX_DATA[0]),
.hps_io_hps_io_emac0_inst_TXD1      (HPS_ENET_TX_DATA[1]),
.hps_io_hps_io_emac0_inst_TXD2      (HPS_ENET_TX_DATA[2]),
.hps_io_hps_io_emac0_inst_TXD3      (HPS_ENET_TX_DATA[3]),
.hps_io_hps_io_emac0_inst_RXD0      (HPS_ENET_RX_DATA[0]),
.hps_io_hps_io_emac0_inst_MDIO      (HPS_ENET_MDIO),
.hps_io_hps_io_emac0_inst_MDC       (HPS_ENET_MDC),
.hps_io_hps_io_emac0_inst_RX_CTL    (HPS_ENET_RX_DV),
.hps_io_hps_io_emac0_inst_TX_CTL    (HPS_ENET_TX_EN),
.hps_io_hps_io_emac0_inst_RX_CLK    (HPS_ENET_RX_CLK),
.hps_io_hps_io_emac0_inst_RXD1      (HPS_ENET_RX_DATA[1]),
.hps_io_hps_io_emac0_inst_RXD2      (HPS_ENET_RX_DATA[2]),
.hps_io_hps_io_emac0_inst_RXD3      (HPS_ENET_RX_DATA[3]),

// SD Card
.hps_io_hps_io_sdio_inst_CMD        (HPS_SD_CMD),
.hps_io_hps_io_sdio_inst_D0         (HPS_SD_DATA[0]),
.hps_io_hps_io_sdio_inst_D1         (HPS_SD_DATA[1]),
.hps_io_hps_io_sdio_inst_CLK        (HPS_SD_CLK),
.hps_io_hps_io_sdio_inst_D2         (HPS_SD_DATA[2]),
.hps_io_hps_io_sdio_inst_D3         (HPS_SD_DATA[3]),

// UART0
.hps_io_hps_io_uart0_inst_RX        (HPS_UART_RX),
.hps_io_hps_io_uart0_inst_TX        (HPS_UART_TX),

// Joystick Inputs (Inputs to HPS)
.pio_buttons_external_connection_export ({js2_sw_clean, js1_sw_clean}),
.pio_js1_x_external_connection_export   (js1_x),
.pio_js1_y_external_connection_export   (js1_y),
.pio_js2_x_external_connection_export   (js2_x),
.pio_js2_y_external_connection_export   (js2_y),

// Servo PWM Outputs (Outputs from HPS)
.pio_pwm0_external_connection_export    (pwm_duty[0]),
.pio_pwm1_external_connection_export    (pwm_duty[1]),
.pio_pwm2_external_connection_export    (pwm_duty[2]),
.pio_pwm3_external_connection_export    (pwm_duty[3]),
```

```
    .pio_pwm4_external_connection_export    (pwm_duty[4]),
    .pio_pwm5_external_connection_export    (pwm_duty[5])
);

// 2. ADC Controller: Reads LTC2308 and feeds joystick channels
adc_controller u_adc (
    .clk        (FPGA_CLK1_50),
    .reset_n    (1'b1),
    .adc_cs_n   (ADC_CS_N),
    .adc_saddr  (ADC_SADDR),
    .adc_sclk   (ADC_SCLK),
    .adc_sdat   (ADC_SDAT),
    .ch0_data   (js1_x),
    .ch1_data   (js1_y),
    .ch2_data   (js2_x),
    .ch3_data   (js2_y)
);

// 3. Debounce Blocks for Joystick Switches
debounce u_db1 (
    .clk       (FPGA_CLK1_50),
    .noisy_in   (JS1_SW),
    .clean_out  (js1_sw_clean)
);

debounce u_db2 (
    .clk        (FPGA_CLK1_50),
    .noisy_in   (JS2_SW),
    .clean_out  (js2_sw_clean)
);

// 4. Six PWM Generators (one per servo)
genvar i;
generate
    for (i = 0; i < 6; i = i + 1) begin : pwm_gen
        pwm_generator #(
            .CLK_FREQ_HZ(50_000_000)
        ) u_pwm (
            .clk                (FPGA_CLK1_50),
            .reset_n            (1'b1),
            .pulse_width_counts (pwm_duty[i]),
            .pwm_out            (GPIO_1[i])
        );
    end
```

```
    endgenerate
endmodule
```

Code Listing: tx_6joint.py - Master Node Telemetry Acquisition and Streaming Script

```
import mmap
import struct
import time
import threading
import urllib2
import json

H2F_BASE = 0xFF200000
H2F_SPAN = 0x1000

JS1_X_OFFSET = 0x60
JS1_Y_OFFSET = 0x70
JS2_X_OFFSET = 0x80
JS2_Y_OFFSET = 0x90
BUTTONS_OFFSET = 0xA0

FIREBASE_URL = "https://de10-iot-bridge-default-rtdb.asia-
southeast1.firebasedatabase.app/teleop.json"

# Joystick Centers
JS1_X_CENTER = 1720
JS1_Y_CENTER = 1738
JS2_X_CENTER = 1690
JS2_Y_CENTER = 1653

DEADBAND = 100

# Initial Angles
base_angle = 90.0
shoulder_angle = 90.0
elbow_angle = 90.0
pitch_angle = 90.0
roll_angle = 90.0
claw_angle = 0.0

previous_btn2 = False
btn1_used_for_roll = False
```

```
# Memory Map Setup
mem_file = open("/dev/mem", "r+b")
mem = mmap.mmap(
    mem_file.fileno(),
    H2F_SPAN,
    mmap.MAP_SHARED,
    mmap.PROT_READ | mmap.PROT_WRITE,
    offset=H2F_BASE
)

def read_u32(offset):
    mem.seek(offset)
    return struct.unpack("<I", mem.read(4))[0]

def calculate_velocity(raw, center):
    diff = raw - center
    if abs(diff) < DEADBAND:
        return 0.0
    speed = (abs(diff) - DEADBAND) / float(4095 - center - DEADBAND)
    speed = min(max(speed, 0.0), 1.0)
    return speed * 1.5

def firebase_upload():
    global base_angle, shoulder_angle, elbow_angle, pitch_angle, roll_angle, claw_angle
    while True:
        try:
            payload = {
                "base": base_angle,
                "shoulder": shoulder_angle,
                "elbow": elbow_angle,
                "wrist_pitch": pitch_angle,
                "wrist_roll": roll_angle,
                "claw": claw_angle,
                "btn1": 0,
                "btn2": 0,
                "timestamp": time.time()
            }
            data = json.dumps(payload)
            request = urllib2.Request(
                FIREBASE_URL,
                data=data,
                headers={"Content-Type": "application/json"}
            )
```

```
            request.get_method = lambda: "PUT"
            urllib2.urlopen(request, timeout=2).read()
        except Exception as e:
            print "\nFirebase error:", e
        time.sleep(0.10)

threading.Thread(target=firebase_upload).start()

print "=========================================="
print "       6-JOINT TELEOP TX STARTED"
print "=========================================="
print "Base      -> JS1 X"
print "Shoulder  -> JS1 Y"
print "Elbow     -> JS2 X"
print "Pitch     -> JS2 Y + BTN1 OFF"
print "Roll      -> JS2 Y + BTN1 ON"
print "Claw      -> BTN2 press/release"
print "=========================================="
print "JS1 X center:", JS1_X_CENTER
print "JS1 Y center:", JS1_Y_CENTER
print "JS2 X center:", JS2_X_CENTER
print "JS2 Y center:", JS2_Y_CENTER
print "=========================================="
print "Initial position: 90 degrees | Claw: CLOSED"
print "=========================================="

last_print = time.time()

try:
    while True:
        raw_base = read_u32(JS1_X_OFFSET)
        raw_shoulder = read_u32(JS1_Y_OFFSET)
        raw_elbow = read_u32(JS2_X_OFFSET)
        raw_pitch_roll = read_u32(JS2_Y_OFFSET)

        buttons = read_u32(BUTTONS_OFFSET)
        btn1 = bool(buttons & 0x01)
        btn2 = bool(buttons & 0x02)

        base_speed = calculate_velocity(raw_base, JS1_X_CENTER)
        shoulder_speed = calculate_velocity(raw_shoulder, JS1_Y_CENTER)
        elbow_speed = calculate_velocity(raw_elbow, JS2_X_CENTER)
        wrist_speed = calculate_velocity(raw_pitch_roll, JS2_Y_CENTER)
```

```
if raw_base > JS1_X_CENTER + DEADBAND:
    base_angle -= base_speed
elif raw_base < JS1_X_CENTER - DEADBAND:
    base_angle += base_speed

if raw_shoulder > JS1_Y_CENTER + DEADBAND:
    shoulder_angle -= shoulder_speed
elif raw_shoulder < JS1_Y_CENTER - DEADBAND:
    shoulder_angle += shoulder_speed

if raw_elbow > JS2_X_CENTER + DEADBAND:
    elbow_angle -= elbow_speed
elif raw_elbow < JS2_X_CENTER - DEADBAND:
    elbow_angle += elbow_speed

if btn1:
    if abs(raw_pitch_roll - JS2_Y_CENTER) >= DEADBAND:
        btn1_used_for_roll = True
        if raw_pitch_roll > (JS2_Y_CENTER + DEADBAND):
            roll_angle -= wrist_speed
        elif raw_pitch_roll < (JS2_Y_CENTER - DEADBAND):
            roll_angle += wrist_speed
else:
    if raw_pitch_roll > (JS2_Y_CENTER + DEADBAND):
        pitch_angle -= wrist_speed
    elif raw_pitch_roll < (JS2_Y_CENTER - DEADBAND):
        pitch_angle += wrist_speed

if previous_btn2 and not btn2:
    if claw_angle == 0.0:
        claw_angle = 180.0
    else:
        claw_angle = 0.0
previous_btn2 = btn2

base_angle = max(0.0, min(180.0, base_angle))
shoulder_angle = max(0.0, min(180.0, shoulder_angle))
elbow_angle = max(0.0, min(180.0, elbow_angle))
pitch_angle = max(0.0, min(180.0, pitch_angle))
roll_angle = max(0.0, min(180.0, roll_angle))
claw_angle = max(0.0, min(180.0, claw_angle))

if time.time() - last_print >= 0.1:
    print "\rBase: %.1f | Shoulder: %.1f | Elbow: %.1f | Pitch: %.1f | Roll: %.1f |
```

```
Claw: %.1f | BTN1: %d | BTN2: %d" % (
            base_angle, shoulder_angle, elbow_angle, pitch_angle, roll_angle, claw_angle,
            1 if btn1 else 0, 1 if btn2 else 0
        ),
        last_print = time.time()

    time.sleep(0.01)

except KeyboardInterrupt:
    print "\n\nTX stopped."

finally:
    mem.close()
    mem_file.close()
```

Code Listing: rx_6joint.py - Slave Node Cloud Polling and Hardware Actuation Script

```
import urllib2
import json
import time
import struct
import mmap

FIREBASE_URL = "https://de10-iot-bridge-default-rtdb.asia-
southeast1.firebasedatabase.app/teleop.json"

H2F_BASE = 0xFF200000
H2F_SPAN = 0x1000

BASE_OFFSET = 0x00
SHOULDER_OFFSET = 0x10
ELBOW_OFFSET = 0x20
WRIST_PITCH_OFFSET = 0x30
WRIST_ROLL_OFFSET = 0x40
CLAW_OFFSET = 0x50

def write_u32(mem, offset, value):
    mem.seek(offset)
    mem.write(struct.pack("<I", int(value)))

def angle_to_pwm(angle, minimum, maximum):
    angle = max(0.0, min(180.0, float(angle)))
```

```
    return int(minimum + (angle / 180.0) * (maximum - minimum))

def get_firebase_data():
    try:
        response = urllib2.urlopen(FIREBASE_URL, timeout=2)
        data = response.read()
        return json.loads(data)
    except Exception as e:
        print "Firebase error:", e
        return None

fd = open("/dev/mem", "r+b")
mem = mmap.mmap(
    fd.fileno(),
    H2F_SPAN,
    mmap.MAP_SHARED,
    mmap.PROT_READ | mmap.PROT_WRITE,
    offset=H2F_BASE
)

print "============================================="
print "       6-JOINT TELEOP RX STARTED"
print "============================================="
print "Base        -> 0x00"
print "Shoulder    -> 0x10"
print "Elbow       -> 0x20"
print "Wrist Pitch -> 0x30"
print "Wrist Roll  -> 0x40"
print "Claw        -> 0x50"
print "============================================="
print "Waiting for Firebase data..."
print "============================================="

last_data = None

try:
    while True:
        data = get_firebase_data()
        if data is not None:
            base = data.get("base", 90.0)
            shoulder = data.get("shoulder", 90.0)
            elbow = data.get("elbow", 90.0)
            pitch = data.get("wrist_pitch", 90.0)
            roll = data.get("wrist_roll", 90.0)
```

```
            claw = data.get("claw", 0.0)

            # ANGLE -> PWM
            base_pwm     = angle_to_pwm(base, 30000, 120000)
            shoulder_pwm = angle_to_pwm(shoulder, 30000, 120000)
            elbow_pwm    = angle_to_pwm(elbow, 30000, 130000)
            pitch_pwm    = angle_to_pwm(pitch, 21000, 119000)
            roll_pwm     = angle_to_pwm(roll, 21000, 119000)
            claw_pwm     = angle_to_pwm(claw, 20000, 60000)

            # WRITE FPGA REGISTERS
            write_u32(mem, BASE_OFFSET, base_pwm)
            write_u32(mem, SHOULDER_OFFSET, shoulder_pwm)
            write_u32(mem, ELBOW_OFFSET, elbow_pwm)
            write_u32(mem, WRIST_PITCH_OFFSET, pitch_pwm)
            write_u32(mem, WRIST_ROLL_OFFSET, roll_pwm)
            write_u32(mem, CLAW_OFFSET, claw_pwm)

            # DISPLAY ONLY WHEN DATA CHANGES
            if data != last_data:
               print ("Base %.1f | Shoulder %.1f | Elbow %.1f | Pitch %.1f | Roll %.1f | Claw
%.1f" %
                     (base, shoulder, elbow, pitch, roll, claw))
               last_data = data

         time.sleep(0.05)

except KeyboardInterrupt:
   print "\nRX stopped."

finally:
   mem.close()
   fd.close()
```

I. PUBLICATIONS

III. Project Team at a Glance



SR. NO.	DETAILS	PHOTOGRAPH
1.	**PROF.(MRS.) P.U. CHATI** (PROJECT GUIDE) CONTACT: - 9890901609 EMAIL: - punam.chati@pcenagpur.edu.in	
2.	**KETAN SHRIRAO** (STUDENT) CONTACT: - 9096586995 EMAIL: - ketanshrirao@gmail.com	
3.	**HEMANT PISE** (STUDENT) CONTACT: - 9579516156 EMAIL: - hemantpise729@gmail.com	
4.	**SAGAR MALLIK** (STUDENT) CONTACT: - 961914065 EMAIL: - skm9167757730@gmail.com	
5.	**AYUSH BOBDE** (STUDENT) CONTACT: - 7058522465 EMAIL: - ayushbobde21@gmail.com	

IV. POSTER

V. DATASHEETS

A. DE-10 Nano Board

Chapter 2
Introduction of the DE10-Nano Board

This chapter provides an introduction to the features and design characteristics of the board.

2.1 Layout and Components

Figure 2-1 and Figure 2-2 shows a photograph of the board. It depicts the layout of the board and indicates the location of the connectors and key components.

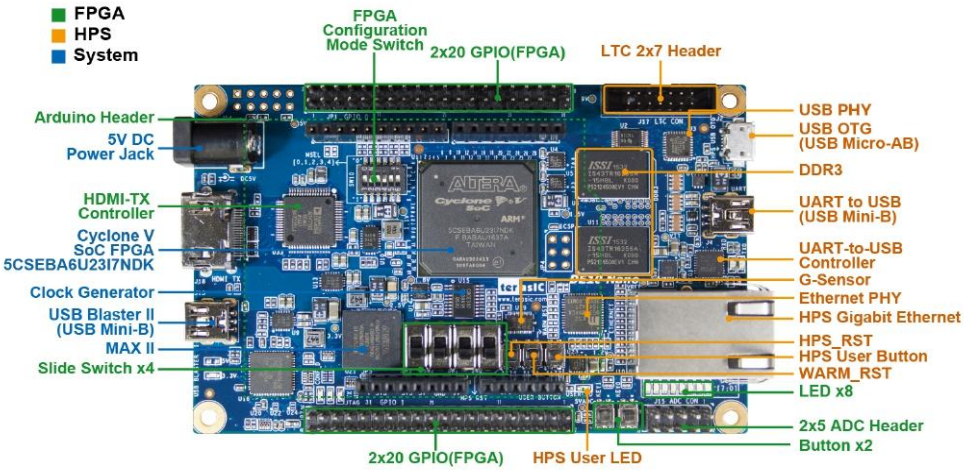


Figure 2-1 DE10-Nano development board (top view)

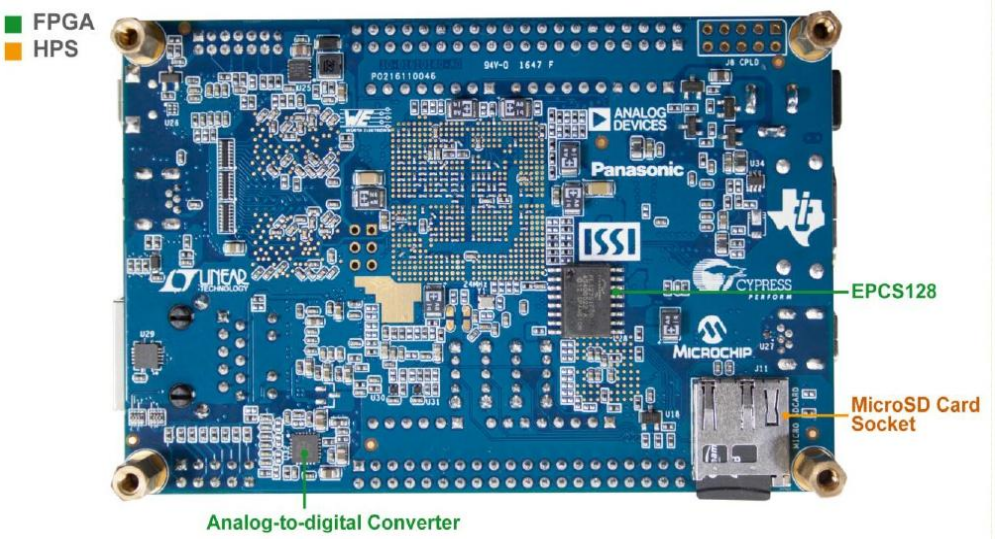


Figure 2-2 DE10-Nano development board (bottom view)

The DE10-Nano board has many features that allow users to implement a wide range of designed circuits, from simple circuits to various multimedia projects.

The following hardware is provided on the board:

■ FPGA

- Cyclone® V SE 5CSEBA6U23I7 device
- Serial configuration device – EPCS128
- USB-Blaster II onboard for programming; JTAG Mode
- 2 push-buttons
- 4 slide switches
- 8 green user LEDs
- Three 50MHz clock sources from the clock generator
- Two 40-pin expansion header
- One Arduino expansion header (Uno R3 compatibility), can connect with Arduino shields.
- One 10-pin Analog input expansion header. (shared with Arduino Analog input)
- A/D converter, 4-wire SPI interface with FPGA
- HDMI TX, compatible with DVI v1.0 and HDCP v1.4

■ HPS (Hard Processor System)

- 800MHz Dual-core ARM Cortex-A9 processor
- 1GB DDR3 SDRAM (32-bit data bus)
- 1 Gigabit Ethernet PHY with RJ45 connector
- port USB OTG, USB Micro-AB connector
- Micro SD card socket
- Accelerometer (I2C interface + interrupt)
- UART to USB, USB Mini-B connector
- Warm reset button and cold reset button
- One user button and one user LED
- LTC 2x7 expansion header

2.2 Block Diagram of the DE10-Nano Board

Figure 2-3 is the block diagram of the board. All the connections are established through the Cyclone V SoC FPGA device to provide maximum flexibility for users. Users can configure the FPGA to implement any system design.

Detailed information about Figure 2-3 are listed below.

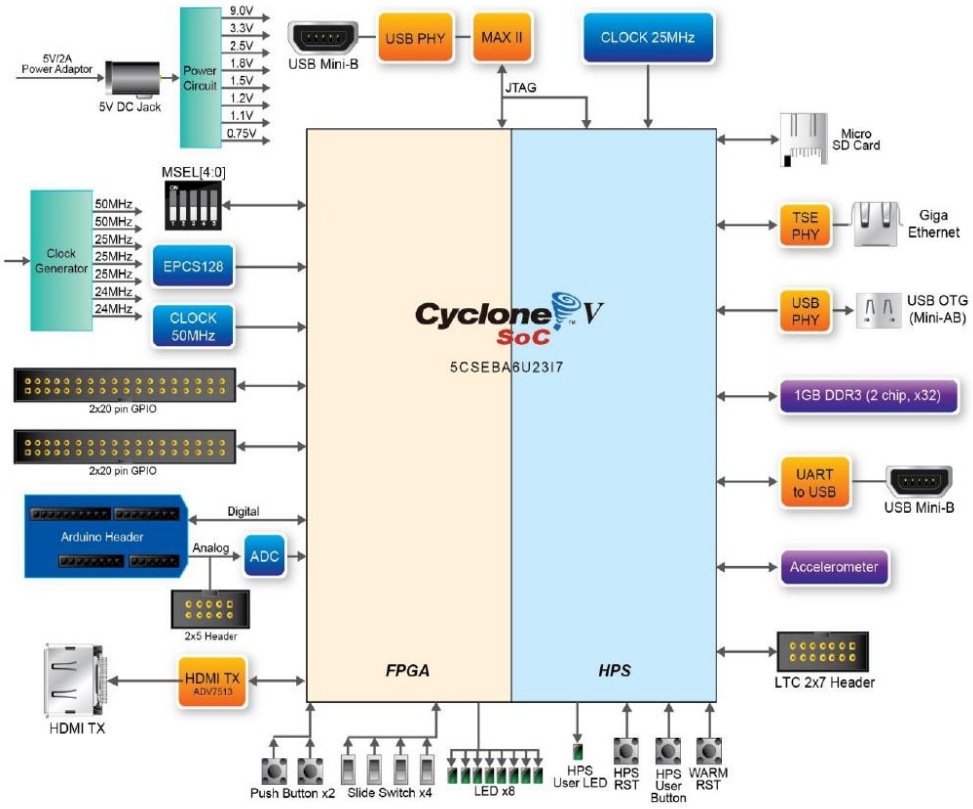


Figure 2-3 Block diagram of DE10-Nano

FPGA Device

- Cyclone V SoC 5CSEBA6U23I7 Device
- Dual-core ARM Cortex-A9 (HPS)
- 110K programmable logic elements
- 5,570 Kbits embedded memory
- 6 fractional PLLs

Configuration and Debug

- Serial configuration device – EPCS128 on FPGA
- Onboard USB-Blaster II (Mini-B USB connector)

Memory Device

- 1GB (2x256Mx16) DDR3 SDRAM on HPS
- Micro SD card socket on HPS

Communication

- One USB 2.0 OTG (ULPI interface with USB Micro-AB connector)
- UART to USB (USB Mini-B connector)
- 10/100/1000 Ethernet

Connectors

- Two 40-pin expansion headers
- Arduino expansion header
- One 10-pin ADC input header
- One LTC connector (one Serial Peripheral Interface (SPI) Master ,one I2C and one GPIO interface)

Display

- HDMI TX, compatible with DVI v1.0 and HDCP v1.4

ADC

- 12-Bit Resolution, 500Ksps Sampling Rate. SPI Interface.
- 8-Channel Analog Input. Input Range : 0V ~ 4.096V.

Chapter 3
Using the DE10-Nano Board

This chapter provides an instruction to use the board and describes the peripherals.

3.1 Settings of FPGA Configuration Mode

When the DE10-Nano board is powered on, the FPGA can be configured from EPCS or HPS.

The MSEL[4:0] pins are used to select the configuration scheme. It is implemented as a 6-pin DIP switch **SW10** on the DE10-Nano board, as shown in Figure 3-1.

Table 3-1 shows the relation between MSEL[4:0] and DIP switch (SW10).

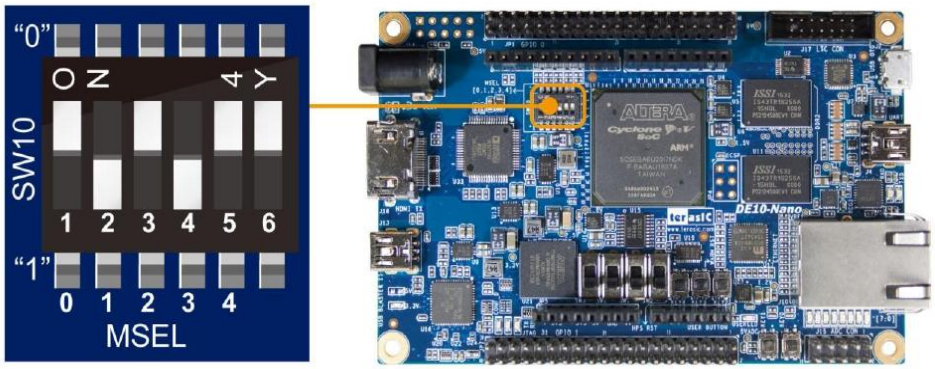


Figure 3-1 DIP switch (SW10) setting of FPPx32 mode.

Table 3-1 FPGA Configuration Mode Switch (SW10)

Board Reference	Signal Name	Description	Default
SW10.1	MSEL0	Use these pins to set the FPGA Configuration scheme	ON ("0")
SW10.2	MSEL1		OFF ("1")
SW10.3	MSEL2		ON ("0")
SW10.4	MSEL3		OFF ("1")
SW10.5	MSEL4		ON ("0")
SW10.6	N/A	N/A	N/A

Table 3-2 shows MSEL[4:0] setting for FPGA configure, and default setting is FPPx32 Mode on DE10-Nano Board.

When the board is powered on and MSEL[4:0] set to "10010", the FPGA is configured from EPCS, which is pre-programmed with the default code. If developers using the "Linux LXDE Desktop" SD Card image, the MSEL[4:0] needs to be set to "01010" before the board is powered on.

Table 3-2 MSEL Pin Settings for FPGA Configure of DE10-Nano Board

Configuration	SW10.1 MSEL0	SW10.2 MSEL1	SW10.3 MSEL2	SW10.4 MSEL3	SW10.5 MSEL4	SW10.6	Description
AS	ON	OFF	ON	ON	OFF	N/A	FPGA configured from EPCS
FPPx32*(1)* / Compression Enabled / Fast POR (Default)	ON	OFF	ON	OFF	ON	N/A	FPGA configured from HPS software: U-Boot, with image stored on the SD card, like LXDE Desktop (default)
FPPx16 / Compression Disabled / Fast POR	ON	ON	ON	ON	ON	N/A	FPGA configured from HPS software: U-Boot, with image stored on the SD card

(1): Please refer to the following link:

https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/hb/cyclone-v/cv_5v4.pdf , See Table 4-1: Configuration Schemes for FPGA Configuration by the HPS



Figure 3-19 Connections between the LEDs and the Cyclone V SoC FPGA

Table 3-6 Pin Assignment of Slide Switches

Signal Name	FPGA Pin No.	Description	I/O Standard
SW[0]	PIN_Y24	Slide Switch[0]	3.3V
SW[1]	PIN_W24	Slide Switch[1]	3.3V
SW[2]	PIN_W21	Slide Switch[2]	3.3V
SW[3]	PIN_W20	Slide Switch[3]	3.3V

Table 3-7 Pin Assignment of Push-buttons

Signal Name	FPGA Pin No.	Description	I/O Standard
KEY[0]	PIN_AH17	Push-button[0]	3.3V
KEY[1]	PIN_AH16	Push-button[1]	3.3V

Table 3-8 Pin Assignment of LEDs

Signal Name	FPGA Pin No.	Description	I/O Standard
LED[0]	PIN_W15	LED [0]	3.3V
LED[1]	PIN_AA24	LED [1]	3.3V
LED[2]	PIN_V16	LED [2]	3.3V
LED[3]	PIN_V15	LED [3]	3.3V
LED[4]	PIN_AF26	LED [4]	3.3V
LED[5]	PIN_AE26	LED [5]	3.3V

3.6.3 Arduino Uno R3 Expansion Header

The board provides Arduino Uno revision 3 compatibility expansion header which comes with four independent headers. The expansion header has 17 user pins (16pins GPIO and 1pin Reset) connected directly to the Cyclone V SoC FPGA. 6-pins Analog input connects to ADC, and also provides DC +9V (VCC9), DC +5V (VCC5), DC +3.3V (VCC3P3 and IOREF), and three GND pins.

Please refer to **Figure 3-21** for detailed pin-out information. The blue font represents the Arduino Uno R3 board pin-out definition.

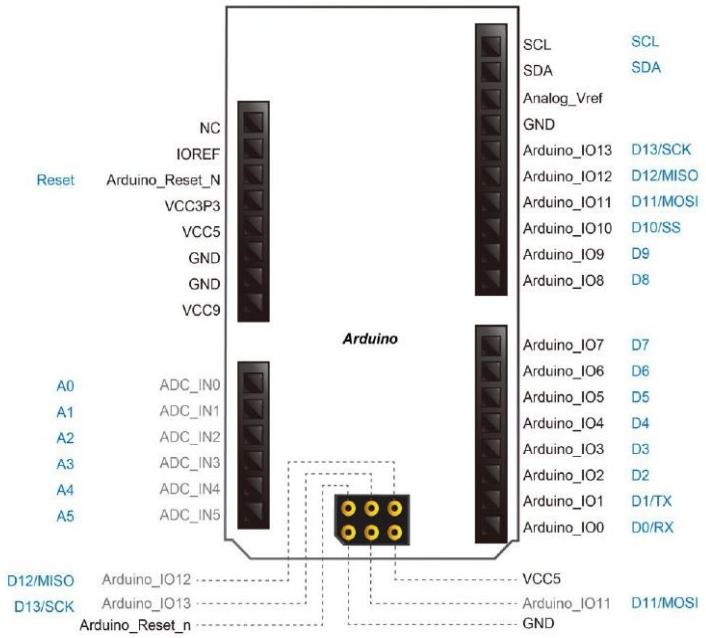


Figure 3-21 lists the all the pin-out signal name of the Arduino Uno connector. The blue font represents the Arduino pin-out definition.

The 16 GPIO pins are provided to the Arduino Header for digital I/O. **Table 3-11** lists the all the pin assignments of the Arduino Uno connector (digital), signal names relative to the Cyclone V SoC FPGA.

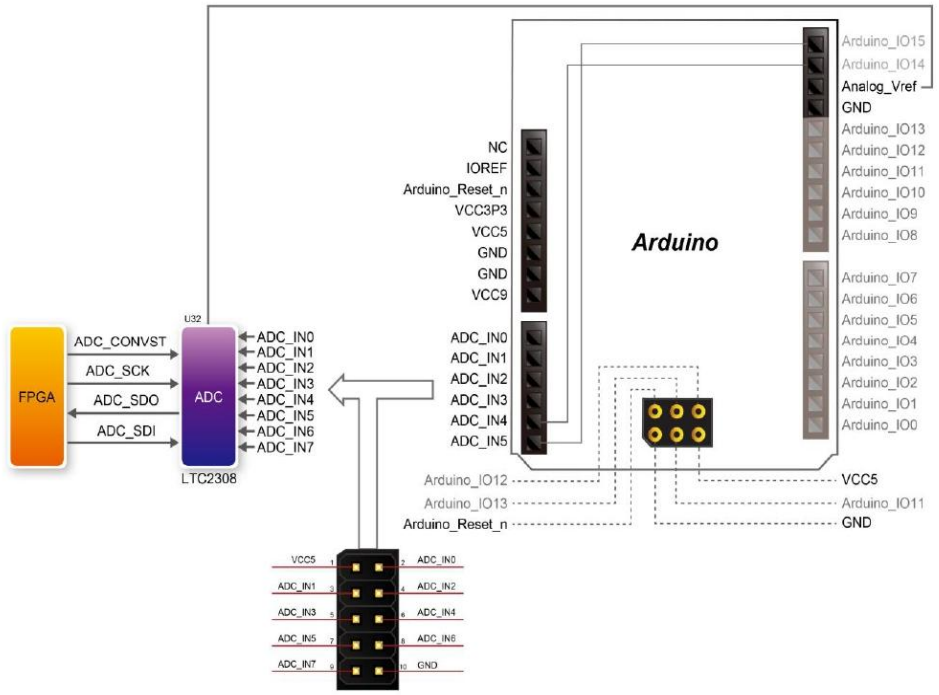


Figure 3-23 Connections between the FPGA, 2x5 header, and the A/D converter

Table 3-12 Pin Assignment of ADC

Signal Name	FPGA Pin No.	Description	I/O Standard
ADC_CONVST	PIN_U9	Conversion Start	3.3V
ADC_SCK	PIN_V10	Serial Data Clock	3.3V
ADC_SDI	PIN_AC4	Serial Data Input (FPGA to ADC)	3.3V
ADC_SDO	PIN_AD4	Serial Data Out (ADC to FPGA)	3.3V

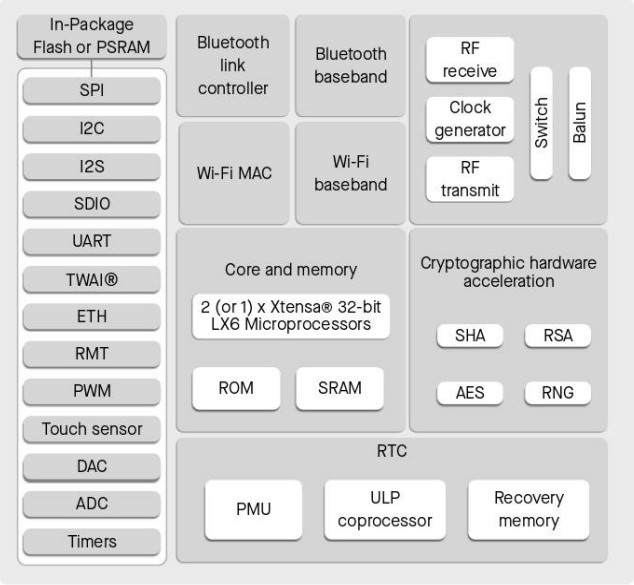
B. ESP-32

Product Overview

ESP32 is a single 2.4 GHz Wi-Fi-and-Bluetooth combo chip designed with the TSMC low-power 40 nm technology. It is designed to achieve the best power and RF performance, showing robustness, versatility and reliability in a wide variety of applications and power scenarios.

For details on part numbers and ordering information, please refer to Section 1 ESP32 Series Comparison. For details on chip revisions, please refer to *ESP32 Chip Revision v3.0 User Guide* and *ESP32 Series SoC Errata*.

The functional block diagram of the SoC is shown below.



ESP32 Functional Block Diagram

Features

Wi-Fi

- 802.11b/g/n
- 802.11n (2.4 GHz), up to 150 Mbps
- WMM
- TX/RX A-MPDU, RX A-MSDU
- Immediate Block ACK
- Defragmentation
- Automatic Beacon monitoring (hardware TSF)
- Four virtual Wi-Fi interfaces
- Simultaneous support for Infrastructure Station, SoftAP, and Promiscuous modes
  Note that when ESP32 is in Station mode, performing a scan, the SoftAP channel will be changed.
- Antenna diversity

Bluetooth®

- Compliant with Bluetooth v4.2 BR/EDR and Bluetooth LE specifications
- Class-1, class-2 and class-3 transmitter without external power amplifier
- Enhanced Power Control
- +9 dBm transmitting power
- NZIF receiver with –94 dBm Bluetooth LE sensitivity
- Adaptive Frequency Hopping (AFH)
- Standard HCI based on SDIO/SPI/UART
- High-speed UART HCI, up to 4 Mbps
- Bluetooth 4.2 BR/EDR and Bluetooth LE dual mode controller
- Synchronous Connection-Oriented/Extended (SCO/eSCO)
- CVSD and SBC for audio codec
- Bluetooth Piconet and Scatternet
- Multi-connections in Classic Bluetooth and Bluetooth LE
- Simultaneous advertising and scanning

CPU and Memory

- Xtensa® single-/dual-core 32-bit LX6 microprocessor(s)
- CoreMark® score:
  - 1 core at 240 MHz: 539.98 CoreMark; 2.25 CoreMark/MHz

- Motor PWM
- LED PWM up to 16 channels

Power Management

- Fine-resolution power control through a selection of clock frequency, duty cycle, Wi-Fi operating modes, and individual power control of internal components
- Five power modes designed for typical scenarios: Active, Modem-sleep, Light-sleep, Deep-sleep, Hibernation
- Power consumption in Deep-sleep mode is 10 μA
- Ultra-Low-Power (ULP) coprocessors
- RTC memory remains powered on in Deep-sleep mode

Security

- Secure boot
- Flash encryption
- 1024-bit OTP, up to 768-bit for customers
- Cryptographic hardware acceleration:
  - AES
  - Hash (SHA-2)
  - RSA
  - Random Number Generator (RNG)

Applications

With low power consumption, ESP32 is an ideal choice for IoT devices in the following areas:

- Smart Home
- Industrial Automation
- Health Care
- Consumer Electronics
- Smart Agriculture
- POS Machines
- Service Robot
- Audio Devices
- Generic Low-power IoT Sensor Hubs
- Generic Low-power IoT Data Loggers
- Cameras for Video Streaming
- Speech Recognition
- Image Recognition
- SDIO Wi-Fi + Bluetooth Networking Card

2 Pins

2.1 Pin Layout

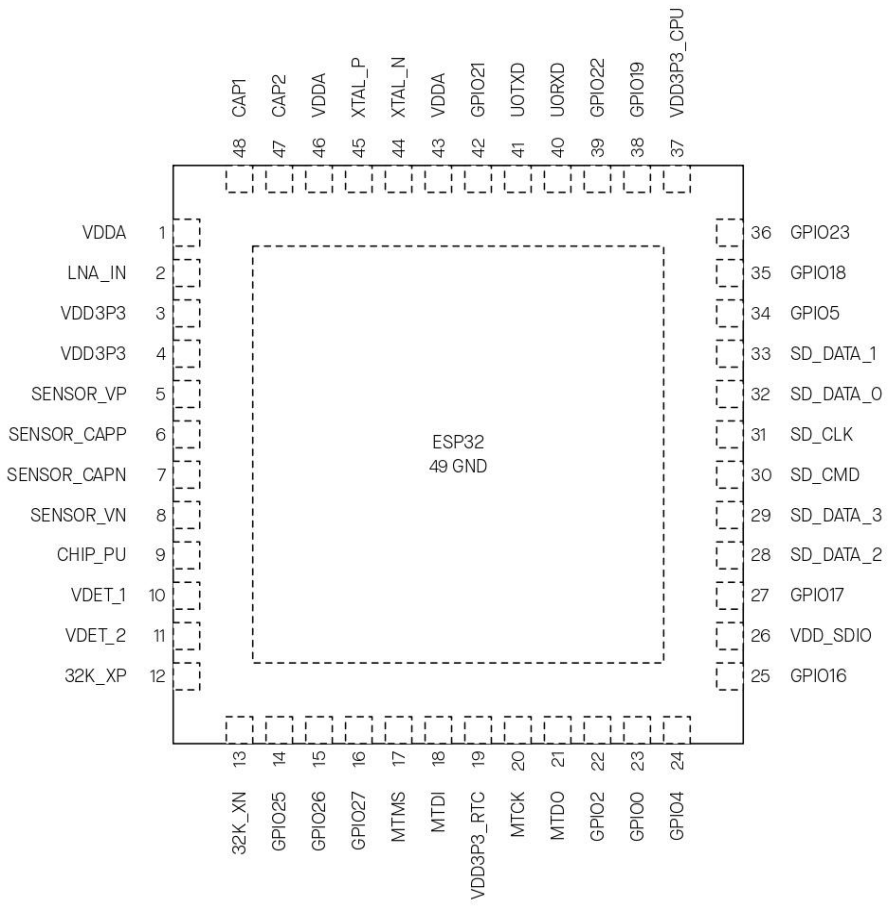


Figure 2-1. ESP32 Pin Layout (QFN 6*6, Top View)

2.2 Pin Overview

Table 2-1. Pin Overview

Name	No.	Type	Function
			Analog
VDDA	1	P	Analog power supply (2.3 V ~ 3.6 V)
LNA_IN	2	I/O	RF input and output
VDD3P3	3	P	Analog power supply (2.3 V ~ 3.6 V)
VDD3P3	4	P	Analog power supply (2.3 V ~ 3.6 V)
			VDD3P3_RTC
SENSOR_VP	5	I	GPIO36, ADC1_CH0, RTC_GPIO0
SENSOR_CAPP	6	I	GPIO37, ADC1_CH1, RTC_GPIO1
SENSOR_CAPN	7	I	GPIO38, ADC1_CH2, RTC_GPIO2
SENSOR_VN	8	I	GPIO39, ADC1_CH3, RTC_GPIO3
CHIP_PU	9	I	High: On; enables the chip Low: Off; the chip shuts down Note: Do not leave the CHIP_PU pin floating.
VDET_1	10	I	GPIO34, ADC1_CH6, RTC_GPIO4
VDET_2	11	I	GPIO35, ADC1_CH7, RTC_GPIO5
32K_XP	12	I/O	GPIO32, ADC1_CH4, RTC_GPIO9, TOUCH9, 32K_XP (32.768 kHz crystal oscillator input)
32K_XN	13	I/O	GPIO33, ADC1_CH5, RTC_GPIO8, TOUCH8, 32K_XN (32.768 kHz crystal oscillator output)
GPIO25	14	I/O	GPIO25, ADC2_CH8, RTC_GPIO6, DAC_1, EMAC_RXD0
GPIO26	15	I/O	GPIO26, ADC2_CH9, RTC_GPIO7, DAC_2, EMAC_RXD1
GPIO27	16	I/O	GPIO27, ADC2_CH7, RTC_GPIO17, TOUCH7, EMAC_RX_DV
MTMS	17	I/O	GPIO14, ADC2_CH6, RTC_GPIO16, TOUCH6, EMAC_TXD2, HSPICLK, HS2_CLK, SD_CLK, MTMS
MTDI	18	I/O	GPIO12, ADC2_CH5, RTC_GPIO15, TOUCH5, EMAC_TXD3, HSPIQ, HS2_DATA2, SD_DATA2, MTDI
VDD3P3_RTC	19	P	Input power supply for RTC IO (2.3 V ~ 3.6 V)
MTCK	20	I/O	GPIO13, ADC2_CH4, RTC_GPIO14, TOUCH4, EMAC_RX_ER, HSPID, HS2_DATA3, SD_DATA3, MTCK
MTDO	21	I/O	GPIO15, ADC2_CH3, RTC_GPIO13, TOUCH3, EMAC_RXD3, HSPICS0, HS2_CMD, SD_CMD, MTDO

Name	No.	Type	Function
GPIO2	22	I/O	GPIO2, ADC2_CH2, RTC_GPIO12, TOUCH2, HSPIWP, HS2_DATA0, SD_DATA0
GPIO0	23	I/O	GPIO0, ADC2_CH1, RTC_GPIO11, TOUCH1, EMAC_TX_CLK, CLK_OUT1,
GPIO4	24	I/O	GPIO4, ADC2_CH0, RTC_GPIO10, TOUCH0, EMAC_TX_ER, HSPIHD, HS2_DATA1, SD_DATA1
			VDD_SDIO
GPIO16	25	I/O	GPIO16, HS1_DATA4, U2RXD, EMAC_CLK_OUT
VDD_SDIO	26	P	Output power supply: 1.8 V or the same voltage as VDD3P3_RTC
GPIO17	27	I/O	GPIO17, HS1_DATA5, U2TXD, EMAC_CLK_OUT_180
SD_DATA_2	28	I/O	GPIO9, HS1_DATA2, U1RXD, SD_DATA2, SPIHD
SD_DATA_3	29	I/O	GPIO10, HS1_DATA3, U1TXD, SD_DATA3, SPIWP
SD_CMD	30	I/O	GPIO11, HS1_CMD, U1RTS, SD_CMD, SPICS0
SD_CLK	31	I/O	GPIO6, HS1_CLK, U1CTS, SD_CLK, SPICLK
SD_DATA_0	32	I/O	GPIO7, HS1_DATA0, U2RTS, SD_DATA0, SPIQ
SD_DATA_1	33	I/O	GPIO8, HS1_DATA1, U2CTS, SD_DATA1, SPID
			VDD3P3_CPU
GPIO5	34	I/O	GPIO5, HS1_DATA6, VSPICS0, EMAC_RX_CLK
GPIO18	35	I/O	GPIO18, HS1_DATA7, VSPICLK
GPIO23	36	I/O	GPIO23, HS1_STROBE, VSPID
VDD3P3_CPU	37	P	Input power supply for CPU IO (1.8 V ~ 3.6 V)
GPIO19	38	I/O	GPIO19, UOCTS, VSPIQ, EMAC_TXD0
GPIO22	39	I/O	GPIO22, UORTS, VSPIWP, EMAC_TXD1
U0RXD	40	I/O	GPIO3, U0RXD, CLK_OUT2
U0TXD	41	I/O	GPIO1, U0TXD, CLK_OUT3, EMAC_RXD2
GPIO21	42	I/O	GPIO21, VSPIHD, EMAC_TX_EN
			Analog
VDDA	43	P	Analog power supply (2.3 V ~ 3.6 V)
XTAL_N	44	O	External crystal output
XTAL_P	45	I	External crystal input
VDDA	46	P	Analog power supply (2.3 V ~ 3.6 V)
CAP2	47	I	Connects to a 3.3 nF (10%) capacitor and 20 kΩ resistor in parallel to CAP1

C. W5500

W5500

The W5500 chip is a Hardwired TCP/IP embedded Ethernet controller that provides easier Internet connection to embedded systems. W5500 enables users to have the Internet connectivity in their applications just by using the single chip in which TCP/IP stack, 10/100 Ethernet MAC and PHY embedded.

WIZnet's Hardwired TCP/IP is the market-proven technology that supports TCP, UDP, IPv4, ICMP, ARP, IGMP, and PPPoE protocols. W5500 embeds the 32Kbyte internal memory buffer for the Ethernet packet processing. If you use W5500, you can implement the Ethernet application just by adding the simple socket program. It's faster and easier way rather than using any other Embedded Ethernet solution. Users can use 8 independent hardware sockets simultaneously.

SPI (Serial Peripheral Interface) is provided for easy integration with the external MCU. The W5500's SPI supports 80 MHz speed and new efficient SPI protocol for the high speed network communication. In order to reduce power consumption of the system, W5500 provides WOL (Wake on LAN) and power down mode.

Features

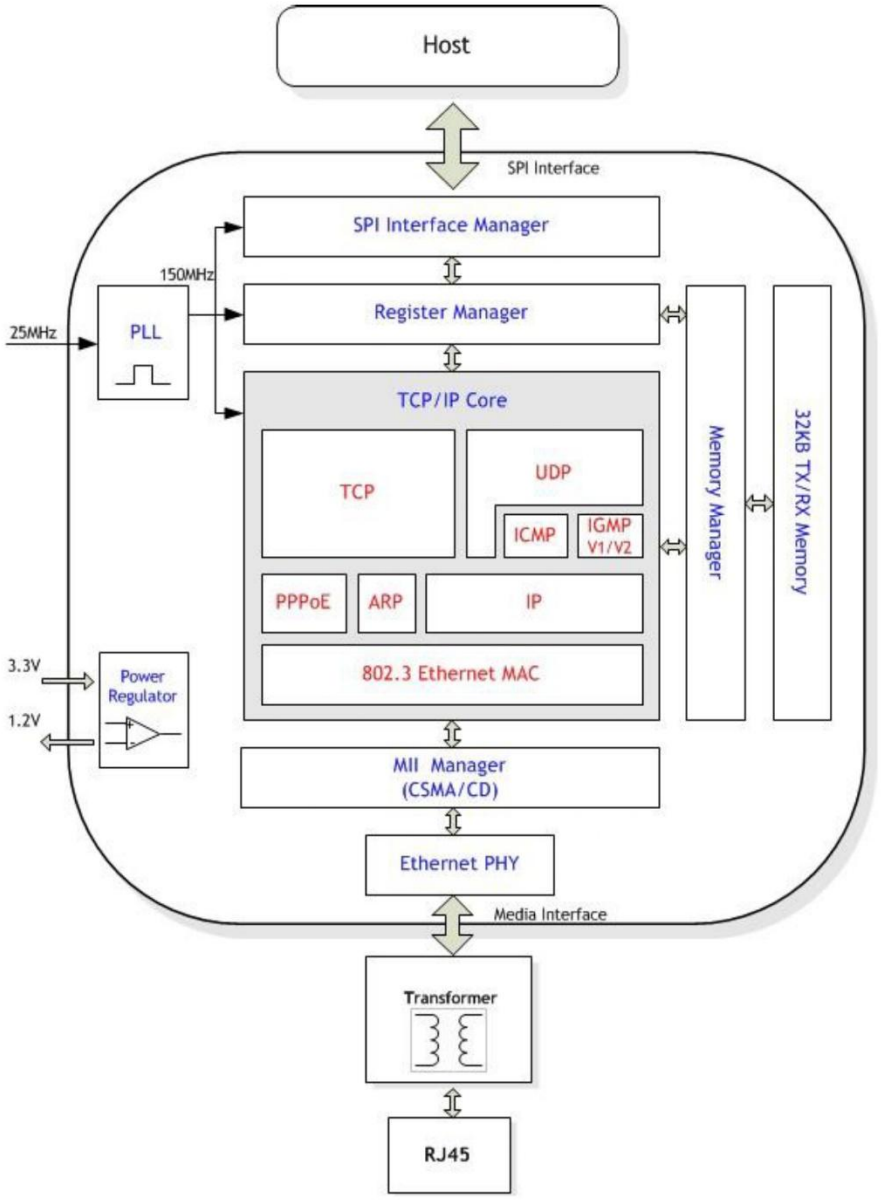
- Supports Hardwired TCP/IP Protocols : TCP, UDP, ICMP, IPv4, ARP, IGMP, PPPoE
- Supports 8 independent sockets simultaneously
- Supports Power down mode
- Supports Wake on LAN over UDP
- Supports High Speed Serial Peripheral Interface(SPI MODE 0, 3)
- Internal 32Kbytes Memory for TX/RX Buffers
- 10BaseT/100BaseTX Ethernet PHY embedded
- Supports Auto Negotiation (Full and half duplex, 10 and 100-based)
- Not supports IP Fragmentation
- 3.3V operation with 5V I/O signal tolerance
- LED outputs (Full/Half duplex, Link, Speed, Active)
- 48 Pin LQFP Lead-Free Package (7x7mm, 0.5mm pitch)

Target Applications

W5500 is suitable for the following embedded applications:

- Home Network Devices: Set-Top Boxes, PVRs, Digital Media Adapters
- Serial-to-Ethernet: Access Controls, LED displays, Wireless AP relays, etc.
- Parallel-to-Ethernet: POS / Mini Printers, Copiers
- USB-to-Ethernet: Storage Devices, Network Printers
- GPIO-to-Ethernet: Home Network Sensors
- Security Systems: DVRs, Network Cameras, Kiosks
- Factory and Building Automations
- Medical Monitoring Equipment
- Embedded Servers

Block Diagram



1 Pin Assignment

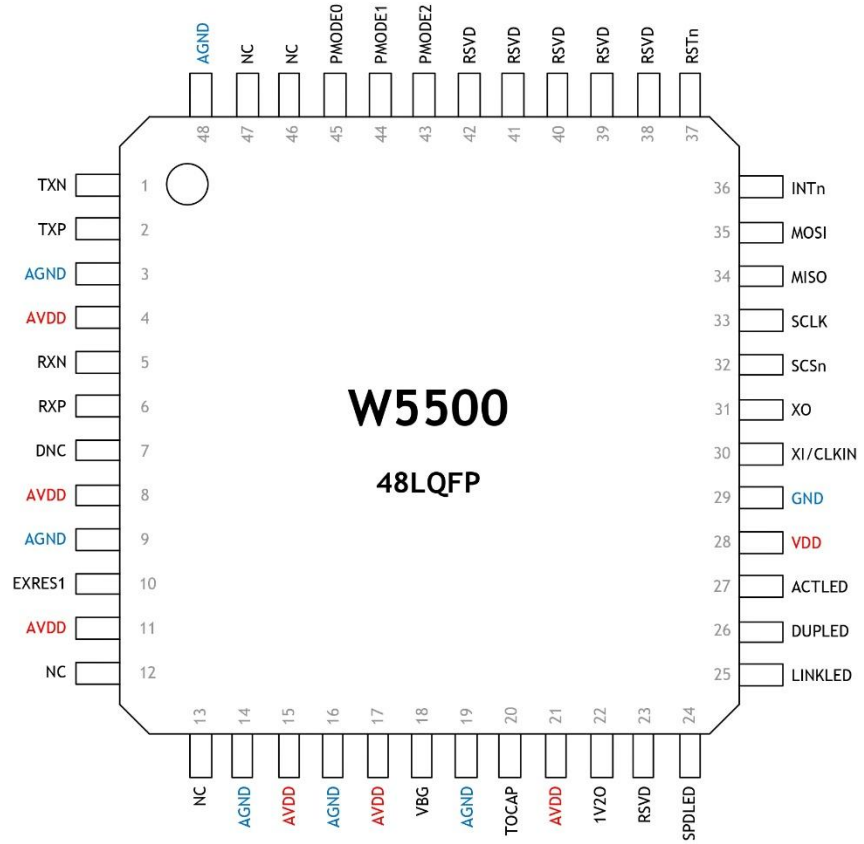


Figure 1. W5500 Pin Layout

1.1 Pin Descriptions

Table 1. Pin Type Notation

Type	Description
I	Input
O	Output
I/O	Input / Output
A	Analog
PWR	3.3V power
GND	Ground

Table 2. W5500 Pin Description

Pin No	Symbol	Internal Bias[1]	Type	Description
1	TXN	-	AO	**TXP/TXN Signal Pair**
2	TXP	-	AO	The differential data is transmitted to the media on the TXP/TXN signal pair.
3	AGND	-	GND	**Analog ground**
4	AVDD	-	PWR	**Analog 3.3V power**
5	RXN	-	AI	**RXP/RXN Signal Pair**
6	RXP	-	AI	The differential data from the media is received on the RXP/RXN signal pair.
7	DNC	-	AI/O	**Do Not Connect Pin**
8	AVDD	-	PWR	**Analog 3.3V power**
9	AGND	-	GND	**Analog ground**
10	EXRES1	-	AI/O	**External Reference Resistor** It should be connected to an external resistor (12.4KΩ, 1%) needed for biasing of internal analog circuits. Refer to the 'External reference resistor' (Figure.2) for details.
11	AVDD	-	PWR	**Analog 3.3V power**
12	-		-	**NC**
13	-		-	**NC**
14	AGND	-	GND	**Analog ground**
15	AVDD	-	PWR	**Analog 3.3V power**
16	AGND	-	GND	**Analog ground**
17	AVDD	-	PWR	**Analog 3.3V power**
18	VBG	-	AO	**Band Gap Output Voltage** This pin will be measured as 1.2V at 25℃. It must be left floating.
19	AGND	-	GND	**Analog ground**
20	TOCAP	-	AO	**External Reference Capacitor** This pin must be connected to a 4.7uF capacitor. The trace length to capacitor should be short to stabilize the internal signals.
21	AVDD	-	PWR	**Analog 3.3V power**
22	1V2O	-	AO	**1.2V Regulator output voltage** This pin must be connected to a 10nF capacitor.

[1] Internal Bias after hardware reset

				This is the output voltage of the internal regulator.
23	RSVD	Pull-down	I	It must be tied to GND.
24	SPDLED	-	O	**Speed LED** This shows the Speed status of the connected link. Low: 100Mbps High: 10Mbps
25	LINKLED	-	O	**Link LED** This shows the Link status. Low: Link is established High: Link is not established
26	DUPLED	-	O	**Duplex LED** This shows the Duplex status for the connected link. Low: Full-duplex mode High: Half-duplex mode
27	ACTLED	-	O	**Active LED** This shows that there is Carrier sense (CRS) from the active Physical Medium Sub-layer (PMD) during TX or RX activity. Low: Carrier sense from the active PMD High: No carrier sense
28	VDD	-	PWR	**Digital 3.3V Power**
29	GND	-	GND	**Digital Ground**
30	XI/CLKIN	-	AI	**Crystal input / External Clock input** External 25MHz Crystal Input. This pin can also be connected to single-ended TTL oscillator (CLKIN). 3.3V clock should be applied for the External Clock input. If this method is implemented, XO should be left unconnected. Refer to the 'Crystal reference schematic' (Figure.3) for details.
31	XO	-	AO	**Crystal output** External 25MHz Crystal Output Note: Float this pin if using an external clock being driven through XI/CLKIN
32	SCSn	Pull-up	I	**Chip Select for SPI bus** This pin can be asserted low to select W5500 in SPI interface. Low: selected High: deselected

<table>
<tr><td>33</td><td>SCLK</td><td>-</td><td>I</td><td>SPI clock input
This pin is used to receive SPI Clock from SPI master.</td></tr>
<tr><td>34</td><td>MISO</td><td>-</td><td>O</td><td>SPI master input slave(W5500) output
When SCSn is Low, this pin outputs SPI data.
When SCSn is High, this pin becomes High Impedance (logically disconnected).</td></tr>
<tr><td>35</td><td>MOSI</td><td>-</td><td>I</td><td>SPI master output slave(W5500) input</td></tr>
<tr><td>36</td><td>INTn</td><td>-</td><td>O</td><td>Interrupt output
(Active low)
Low: Interrupt asserted from W5500
High: No interrupt</td></tr>
<tr><td>37</td><td>RSTn</td><td>Pull-up</td><td>I</td><td>Reset
(Active low)
RESET should be held low at least 500 us for W5500 reset.</td></tr>
<tr><td>38</td><td>RSVD</td><td>Pull-down</td><td>I</td><td>NC</td></tr>
<tr><td>39</td><td>RSVD</td><td>Pull-down</td><td>I</td><td>NC</td></tr>
<tr><td>40</td><td>RSVD</td><td>Pull-down</td><td>I</td><td>NC</td></tr>
<tr><td>41</td><td>RSVD</td><td>Pull-down</td><td>I</td><td>NC</td></tr>
<tr><td>42</td><td>RSVD</td><td>Pull-down</td><td>I</td><td>NC</td></tr>
<tr><td>43</td><td>PMODE2</td><td>Pull-up</td><td>I</td><td rowspan="3">PHY Operation mode select pins
These pins determine the network mode. Refer to the below table for details.
<table>
<tr><th colspan="3">PMODE [2:0]</th><th rowspan="2">Description</th></tr>
<tr><th>2</th><th>1</th><th>0</th></tr>
<tr><td>0</td><td>0</td><td>0</td><td>10BT Half-duplex, Auto-negotiation disabled</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>10BT Full-duplex, Auto-negotiation disabled</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>100BT Half-duplex, Auto-negotiation disabled</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>100BT Full-duplex, Auto-negotiation disabled</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>100BT Half-duplex, Auto-negotiation enabled</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>Not used</td></tr>
<tr><td>1</td><td>1</td><td>0</td><td>Not used</td></tr>
<tr><td>1</td><td>1</td><td>1</td><td>All capable, Auto-negotiation enabled</td></tr>
</table></td></tr>
<tr><td>44</td><td>PMODE1</td><td>Pull-up</td><td>I</td></tr>
<tr><td>45</td><td>PMODE0</td><td>Pull-up</td><td>I</td></tr>
<tr><td>46</td><td>-</td><td>-</td><td>-</td><td>NC</td></tr>
<tr><td>47</td><td>-</td><td>-</td><td>-</td><td>NC</td></tr>
<tr><td>48</td><td>AGND</td><td>-</td><td>GND</td><td>Analog ground</td></tr>
</table>

Priyadarshini College of Engineering, Nagpur

Department of Electronics & Telecommunication Engineering

Final Year Project

Year of Study: 2026-2027 (7th Sem) (Section-A)

Title of Project: Design and Implementation of Cloud based Robotic ARM Teleoperation system using FPGA SoC.

Course Outcomes (COs)

- **CO1:** Identify and define a complex engineering problem with clear project objectives and scope, considering societal and environmental aspects.
- **CO2:** Apply engineering knowledge, modern tools, and relevant technologies to design, develop, and implement an effective solution or system.
- **CO3:** Analyze and interpret data/results critically to evaluate the performance of the proposed solution using appropriate validation techniques.
- **CO4:** Work effectively as an individual and as a team member, demonstrating professional ethics, communication skills, and project management abilities.
- **CO5:** Prepare and deliver technical documentation and oral presentations with clarity and precision, adhering to engineering standards and practices.
- **CO6:** Reflect on lifelong learning by identifying gaps in knowledge, exploring recent advancements, and proposing further improvements in the project.

Program Outcomes (POs)

1. **Engineering knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8. **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes (PSOs)

- **PSO1:** To identify, formulate and analyze the problems in Electronics & Telecommunication Engineering by using principles of Mathematics and engineering fundamentals.
- **PSO2:** Applying Electronic and Communication Engineering knowledge for design and analysis of various system.

CO-PO & PSO Mapping Matrix

Subject	Course Outcomes	Program Outcomes (POs)											PSOs	
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
Final Year Project	**CO-1** Identify and define a complex engineering problem with clear project objectives and scope, considering societal and environmental aspects.		3				2						3	
	CO-2 Apply engineering knowledge, modern tools, and relevant technologies to design, develop, and implement an effective solution or system.	3		3		2								3
	CO-3 Analyze and interpret data/results critically to evaluate the performance of the proposed solution using appropriate validation techniques.				3								1	2
	CO-4 Work effectively as an individual and as a team member, demonstrating professional ethics, communication skills, and project management abilities.							2	3	2	2			
	CO-5 Prepare and deliver technical documentation and oral presentations with clarity and precision, adhering to engineering standards and practices.									3				

Subject	Course Outcomes	Program Outcomes (POs)											PSOs	
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
	CO-6 Reflect on lifelong learning by identifying gaps in knowledge, exploring recent advancements, and proposing further improvements in project.											3		

V. Plagiarism Report

Report provided by the «eAarjav» service
http://pcenagpur.eaarjav.com

Check Report

Author: Submitted Project Report
Document: THESIS A1

Checked by: Kadbe Mahesh
Organization: Priyadarshini College of Engineering, Nagpur

CHECK RESULTS



Matches: 9.92%
Originality: 90.08%
Strong AI Indicators: 0%
Quotes: 0%
Text recycling: 0%

Matches, Text Reuse, Text Recycling, and Originality are individual indicators displayed as percentages. Their sum is equal to 100%, which is the entire text of the checked document.

Checked: 97.96% of document text, exclude from check: 2.04% of document text. Sections and elements disabled by the user: Bibliography

- **Matches** — segments in the checked text that are fully or partially similar to identified sources, except for those that the system has classified as text reuse or recycling. The Matches value reflects the share of the checked text segments classified as matches in the overall text volume.
- **Text recycling** — includes segments in the checked text that are identical or nearly identical to a source text fragment whose author or co-author is the author of the document being checked. The Text Recycling value reflects the share of the checked text segments classified as text recycling in the overall text volume.
- **Quotes** — includes segments in the checked text that are not original, but which the system considers correctly formulated. Text reuse also includes boilerplate phrases, bibliographies, and text segments found by the Garant Legal Information System: Regulatory Documents search module. The Text Reuse value reflects the share of the checked text segments classified as text reuse in the overall text volume.
- **Text crossing** — text fragment of a checked document which is identical or almost identical to a fragment of the source text.
- **Source** — document indexed by the system and contained in the search module which is used for the check.
- **Original text** — includes segments in the checked text that are not found in any source or tagged by any of the search modules. The Originality value reflects the share of the checked text segments classified as original text in the overall text volume.

Please note that the system finds overlapping texts in the checked document and text sources indexed by the system. At the same time, the system is an auxiliary tool. Correctness and adequacy of reuse or quotes, as well as authorship of text fragments in the checked document must be determined by the verifier.

DOCUMENT INFORMATION

Document No.: 1726
Document type: Not specified
Check start: 16.10.2026 12:48:11
Date of correction: No

Number of pages: 108
Number of characters: 95655
Number of words: 13928
Number of sentences: 1238

Comment: not specified

REPORT INFORMATION

Check performed considering editing: Yes

Text recognition performed (OCR): No

Check performed considering structure: Yes

Sections and elements disabled by the user: Bibliography

Search modules: Citations, Consolidated collection of scientific works of Belarus, Media collection, eLIBRARY publications collection, Cross-language search for text reuse from open publications of international publishers, ELS joint collection, GARANT legal documentation, Springer Nature, RSL publications collection, Cross language English Internet, Legal information system 'Adilet' search module, GARANT analytics, IEEE, Cross-language search for reuse from PubMed open access publication database, Institutes Unity collection (cross language search and paraphrases), Collection of the National Library of Uzbekistan, Bibliography separation module, Patents collection, IEEE Cross language, Search module of paraphrases from open publications of international publishers, eLIBRARY publications (translations and paraphrases), Search module of IEEE paraphrases, Search module of PubMed paraphrases, Global Unity Collection, Search module of common phrases, Institutes Unity collection, PubMed, Publisher OA Collection, INVENTORUS Collection, RSL publications (translations and paraphrases), Search module of English Internet paraphrases, Search module Internet Plus Online, Company's own collection, Company's own collection (cross language search and paraphrases)

SOURCES

№	Text share	Report share	Source	Valid from	Search module
[01]	3.02%	2.9%	https://ksrce.ac.in/admin/file_manager/source/New/Academic/curriculumSyllabi/... https://ksrce.ac.in	05 Jul 2025	Search module Internet Plus Online
[02]	2.61%	1.85%	https://nemertes.library.upatras.gr/server/api/core/bitstreams/852eec1d-be7a-4c... https://nemertes.library.upatras.gr	30 Dec 2022	Search module Internet Plus Online
[03]	1.02%	1.02%	Interlink Platform for School, Higher and Technical Education in India: Design Plat... https://platform.inventorus.ru	13 Apr 2025	INVENTORUS Collection
[04]	3.37%	0.98%	https://presidencyuniversity.in/uploads/images/680f5bd7b1d681745837015.pdf https://presidencyuniversity.in	19 Jun 2026	Search module of English Internet paraphrases
[05]	2.78%	0.93%	https://teses.usp.br/teses/disponiveis/18/18152/tde-23072021-152559/publico/Di... https://teses.usp.br	05 Jan 2023	Search module Internet Plus Online
[06]	0.92%	0.92%	AQAR-23-24.pdf https://pcenagpur.edu.in	15 Apr 2026	Search module Internet Plus Online
[07]	0.24%	0.24%	Task Decision-Making and Obstacle Clearing Optimization of Police Agents in Rob... https://ieeexplore.ieee.org	20 Jun 2025	IEEE
[08]	0.24%	0.24%	GROUP 16 SEM 8 MAJOR PROJECT	24 Apr 2024	Institutes Unity collection (cross language search and paraphrases)
[09]	0.21%	0.21%	Energy Consumption Monitoring of FPGA Controller https://ieeexplore.ieee.org	30 Jan 2023	IEEE
[10]	0.17%	0.17%	Distribution Feeder Parameter Estimation Without Synchronized Phasor Measure... https://ieeexplore.ieee.org	04 Jan 2022	IEEE
[11]	0.15%	0.15%	Дугаев, Дмитрий Александрович Разработка адаптивного алгоритма маршр... http://dlib.rsl.ru	01 Jan 2017	RSL publications collection
[12]	2.17%	0.15%	https://ksrce.ac.in/admin/uploads/departments/achievements/magazine/8/9372_... https://ksrce.ac.in	12 Jul 2025	Search module Internet Plus Online
[13]	1.87%	0.1%	Full final draft 3 report	22 Apr 2024	Institutes Unity collection
[14]	0.13%	0.06%	GROUP 16 SEM 8 MAJOR PROJECT	24 Apr 2024	Institutes Unity collection
[15]	1.93%	0%	Towards a machine learning-based constructive alignment approach for improvin... https://link.springer.com	01 May 2024	Springer Nature
[16]	2.22%	0%	Constructively Aligning Deliveries and Assessment to Sustainable Development G... https://ieeexplore.ieee.org	25 Feb 2026	IEEE
[17]	2.05%	0%	The Complexity of Graduate Attributes in Engineering for Industry 4.0 Contexts https://ieeexplore.ieee.org	13 Dec 2023	IEEE
[18]	2.03%	0%	Automated Alignment of Course Outcomes with Program Outcomes and Progra...	13 Oct 2025	IEEE
[19]	2.01%	0%	Faculty of Engineering https://ieeexplore.ieee.org	23 Feb 2026	IEEE
[20]	1.14%	0%	Isn't it Time We Told the Truth About Engineering Soft Skills? https://ieeexplore.ieee.org	01 Dec 2025	IEEE
[21]	1.01%	0%	Full final draft 3 report	22 Apr 2024	Institutes Unity collection (cross

					language search and paraphrases)
[22]	2.23%	0%	Major_project_report_p1(1)	22 Apr 2024	Institutes Unity collection (cross language search and paraphrases)
[23]	1.58%	0%	grp_14_image_mosaicing_draft_1	22 Apr 2024	Institutes Unity collection (cross language search and paraphrases)
[24]	0.23%	0%	Multimedia interface having a multimedia processor and a field programmable g... http://freepatentsonline.com	06 Nov 2016	Patents collection
[25]	0.23%	0%	Method and apparatus for significantly improving the reliability of multilevel me... http://freepatentsonline.com	06 Nov 2016	Patents collection
[26]	0.22%	0%	DRAM enhanced processor - Kawasaki Microelectronics, Inc. http://freepatentsonline.com	06 Nov 2016	Patents collection
[27]	0.67%	0%	Open Distance Learning of Modern Engineering Tool and Its Assessment for Civil ... https://openalex.org	04 Jan 2023	Search module of paraphrases from open publications of international publishers
[28]	0.25%	0%	Student writing in the engineering curriculum: discursive rhetorical model of the l... https://openalex.org	15 Nov 2023	Search module of paraphrases from open publications of international publishers
[29]	1.89%	0%	Компетенции выпускников инженерных программ: национальные и междун...	27 Apr 2025	eLIBRARY publications (translations and paraphrases)
[30]	1.67%	0%	Профессиональная подготовка студентов в системе высшего технического о... http://elibrary.ru	01 Jan 2016	eLIBRARY publications (translations and paraphrases)
[31]	1.32%	0%	Образование, ориентированное на результат, в Пакистане: путь к обеспече...	03 Dec 2025	eLIBRARY publications (translations and paraphrases)
[32]	0.32%	0%	Comparative Study of Outcome Based Accreditation: NBA 2016 vs. 2024 Program ... https://ieeexplore.ieee.org	02 Jun 2025	Search module of IEEE paraphrases
[33]	2.17%	0%	Isn't it Time We Told the Truth About Engineering Soft Skills? https://ieeexplore.ieee.org	01 Dec 2025	Search module of IEEE paraphrases
[34]	2.56%	0%	Automated Alignment of Course Outcomes with Program Outcomes and Progra...	13 Oct 2025	Search module of IEEE paraphrases
[35]	0.76%	0%	Constructively Aligning Deliveries and Assessment to Sustainable Development G... https://ieeexplore.ieee.org	25 Feb 2026	Search module of IEEE paraphrases
[36]	2.33%	0%	Beyond Assessment: Measuring and Developing Graduate Attributes for the Engi...	25 Aug 2025	Search module of IEEE paraphrases
[37]	2.65%	0%	Faculty of Engineering https://ieeexplore.ieee.org	23 Feb 2026	Search module of IEEE paraphrases
[38]	1.8%	0%	Major_project_report_p1(1)	22 Apr 2024	Institutes Unity collection
[39]	1.77%	0%	grp_14_image_mosaicing_draft_1	22 Apr 2024	Institutes Unity collection
[40]	0.23%	0%	A Standard-Based Internet of Things Platform and Data Flow Modeling for Smart ... https://pmc.ncbi.nlm.nih.gov	20 Jun 2021	PubMed
[41]	0.2%	0%	Student writing in the engineering curriculum: discursive rhetorical model of the l... https://openalex.org	15 Nov 2023	Publisher OA Collection
[42]	0.16%	0%	The role of Baitulmal towards the education of poor muslims in Sarawak https://openalex.org	01 Jan 2012	Publisher OA Collection
[43]	0.14%	0%	Development of Smart, Compact Fusion Diagnostics using Field-Programmable G... https://openalex.org	01 Jan 2017	Publisher OA Collection
[44]	2%	0%	Assessment of outcome-based integrated design project https://platform.inventorus.ru	01 Feb 2019	INVENTORUS Collection
[45]	1.55%	0%	Addressing Generic Skills Gap for Future Civil Engineers in The 4th Industrial Revo... https://platform.inventorus.ru	15 Mar 2024	INVENTORUS Collection
[46]	1.54%	0%	The Integration of Environmental Educational Outcomes within Higher Education ... https://platform.inventorus.ru	25 Dec 2012	INVENTORUS Collection
[47]	1.24%	0%	Digital competences of the industrial engineer in industry 4.0 a systematic vision https://platform.inventorus.ru	01 Jan 2022	INVENTORUS Collection
[48]	0.62%	0%	Online Mode of Teaching and Learning Process in Engineering Discipline: Teacher... https://platform.inventorus.ru	13 Feb 2023	INVENTORUS Collection
[49]	0.17%	0%	Network Latency in Teleoperation of Connected and Autonomous Vehicles: A Revi... https://platform.inventorus.ru	18 Jun 2024	INVENTORUS Collection
[50]	0.21%	0%	IoT-based on boiler fuel monitoring system: a case of Raha beverages company li... https://platform.inventorus.ru	01 Aug 2023	INVENTORUS Collection
[51]	0.1%	0%	Intelligent Secure Trustable Things https://platform.inventorus.ru	01 Jan 2024	INVENTORUS Collection